

Sudoku

Wygenerowano za pomocą Doxygen 1.16.1

1 Struktura katalogów	1
1.1 Katalogi	1
2 Indeks hierarchiczny	3
2.1 Hierarchia klas	3
3 Indeks klas	5
3.1 Lista klas	5
4 Indeks plików	7
4.1 Lista plików	7
5 Dokumentacja katalogów	9
5.1 Dokumentacja katalogu core	9
6 Dokumentacja klas	11
6.1 Dokumentacja klasy Board	11
6.1.1 Opis szczegółowy	12
6.1.2 Dokumentacja konstruktora i destruktora	12
6.1.2.1 Board()	12
6.1.3 Dokumentacja funkcji składowych	12
6.1.3.1 at() [1/2]	12
6.1.3.2 at() [2/2]	13
6.1.3.3 clear()	13
6.1.3.4 getValue()	13
6.1.3.5 isEmpty()	14
6.1.3.6 isFixed()	15
6.1.3.7 isFull()	16
6.1.3.8 operator() [1/2]	17
6.1.3.9 operator() [2/2]	17
6.1.3.10 setValue()	18
6.1.4 Dokumentacja atrybutów składowych	19
6.1.4.1 BOX_SIZE	19
6.1.4.2 SIZE	19
6.2 Dokumentacja klasy Cell	19
6.2.1 Opis szczegółowy	20
6.2.2 Dokumentacja konstruktora i destruktora	20
6.2.2.1 Cell() [1/2]	20
6.2.2.2 Cell() [2/2]	20
6.2.3 Dokumentacja funkcji składowych	20
6.2.3.1 clear()	20
6.2.3.2 getValue()	21
6.2.3.3 isEmpty()	21
6.2.3.4 isFixed()	22

6.2.3.5 setFixed()	22
6.2.3.6 setValue()	22
6.3 Dokumentacja klasy ConsoleUI	23
6.3.1 Opis szczegółowy	23
6.3.2 Dokumentacja konstruktora i destruktora	23
6.3.2.1 ConsoleUI()	23
6.3.3 Dokumentacja funkcji składowych	24
6.3.3.1 run()	24
6.4 Dokumentacja klasy GameMode	24
6.4.1 Opis szczegółowy	25
6.4.2 Dokumentacja konstruktora i destruktora	25
6.4.2.1 ~GameMode()	25
6.4.3 Dokumentacja funkcji składowych	26
6.4.3.1 clearCell()	26
6.4.3.2 getBoxSize()	26
6.4.3.3 getDifficulty()	27
6.4.3.4 getMaxValue()	27
6.4.3.5 getName()	27
6.4.3.6 getSize()	28
6.4.3.7 getValue()	28
6.4.3.8 isCompleted()	29
6.4.3.9 isFixed()	29
6.4.3.10 loadState()	30
6.4.3.11 makeMove()	30
6.4.3.12 startNew()	31
6.5 Dokumentacja klasy Generator	31
6.5.1 Opis szczegółowy	31
6.5.2 Dokumentacja funkcji składowych	31
6.5.2.1 generatePuzzle()	31
6.6 Dokumentacja klasy LargeSudokuMode	32
6.6.1 Opis szczegółowy	34
6.6.2 Dokumentacja konstruktora i destruktora	34
6.6.2.1 LargeSudokuMode()	34
6.6.3 Dokumentacja funkcji składowych	35
6.6.3.1 getName()	35
6.7 Dokumentacja klasy Position	35
6.7.1 Opis szczegółowy	36
6.7.2 Dokumentacja konstruktora i destruktora	36
6.7.2.1 Position()	36
6.7.3 Dokumentacja funkcji składowych	36
6.7.3.1 getCol()	36
6.7.3.2 getRow()	37

6.7.3.3 operator!=()	37
6.7.3.4 operator==()	37
6.7.3.5 set()	38
6.7.4 Dokumentacja przyjaciół i powiązanych symboli	38
6.7.4.1 operator<<	38
6.8 Dokumentacja klasy SaveManager	39
6.8.1 Opis szczegółowy	39
6.8.2 Dokumentacja funkcji składowych	39
6.8.2.1 load()	39
6.8.2.2 save()	40
6.8.3 Dokumentacja atrybutów składowych	40
6.8.3.1 DEFAULT_SAVE_FILE	40
6.9 Dokumentacja klasy Solver	41
6.9.1 Opis szczegółowy	41
6.9.2 Dokumentacja funkcji składowych	41
6.9.2.1 solve()	41
6.10 Dokumentacja klasy StandardSudokuMode	42
6.10.1 Opis szczegółowy	44
6.10.2 Dokumentacja konstruktora i destruktora	44
6.10.2.1 StandardSudokuMode()	44
6.10.3 Dokumentacja funkcji składowych	45
6.10.3.1 getName()	45
6.11 Dokumentacja klasy SudokuGame	45
6.11.1 Opis szczegółowy	45
6.11.2 Dokumentacja konstruktora i destruktora	46
6.11.2.1 SudokuGame()	46
6.11.3 Dokumentacja funkcji składowych	46
6.11.3.1 clearCell()	46
6.11.3.2 getBoard()	46
6.11.3.3 isCompleted()	47
6.11.3.4 makeMove()	47
6.11.3.5 newGame()	48
6.12 Dokumentacja klasy SudokuModeBase	49
6.12.1 Opis szczegółowy	50
6.12.2 Dokumentacja konstruktora i destruktora	51
6.12.2.1 SudokuModeBase()	51
6.12.3 Dokumentacja funkcji składowych	51
6.12.3.1 canPlaceValue()	51
6.12.3.2 clearCell()	52
6.12.3.3 emptyCellsForDifficulty()	53
6.12.3.4 generateSolvedValues()	53
6.12.3.5 getBoxSize()	54

6.12.3.6	getDifficulty()	54
6.12.3.7	getMaxValue()	54
6.12.3.8	getSize()	55
6.12.3.9	getValue()	55
6.12.3.10	index()	56
6.12.3.11	isCompleted()	56
6.12.3.12	isFixed()	57
6.12.3.13	isInside()	58
6.12.3.14	loadState()	58
6.12.3.15	makeMove()	59
6.12.3.16	startNew()	59
6.12.4	Dokumentacja atrybutów składowych	60
6.12.4.1	boxSize	60
6.12.4.2	difficulty	60
6.12.4.3	fixedFields	60
6.12.4.4	size	60
6.12.4.5	values	60
6.13	Dokumentacja klasy SudokuRules	60
6.13.1	Opis szczegółowy	61
6.13.2	Dokumentacja funkcji składowych	61
6.13.2.1	isBoardValid()	61
6.13.2.2	isMoveValid()	62
6.13.2.3	isValueInBox()	63
6.13.2.4	isValueInColumn()	64
6.13.2.5	isValueInRow()	65
7	Dokumentacja plików	67
7.1	Dokumentacja pliku core/Board.cpp	67
7.2	Dokumentacja pliku core/Board.h	67
7.3	Board.h	68
7.4	Dokumentacja pliku core/Cell.cpp	69
7.5	Dokumentacja pliku core/Cell.h	69
7.6	Cell.h	70
7.7	Dokumentacja pliku core/ConsoleUI.cpp	70
7.8	Dokumentacja pliku core/ConsoleUI.h	71
7.9	ConsoleUI.h	72
7.10	Dokumentacja pliku core/GameMode.cpp	73
7.10.1	Dokumentacja funkcji	73
7.10.1.1	difficultyFromString()	73
7.10.1.2	difficultyToString()	74
7.11	Dokumentacja pliku core/GameMode.h	75
7.11.1	Dokumentacja typów wyliczanych	76

7.11.1.1 Difficulty	76
7.11.2 Dokumentacja funkcji	76
7.11.2.1 difficultyFromString()	76
7.11.2.2 difficultyToString()	77
7.12 GameMode.h	77
7.13 Dokumentacja pliku core/Generator.cpp	78
7.14 Dokumentacja pliku core/Generator.h	79
7.15 Generator.h	79
7.16 Dokumentacja pliku core/LargeSudokuMode.cpp	80
7.17 Dokumentacja pliku core/LargeSudokuMode.h	81
7.18 LargeSudokuMode.h	81
7.19 Dokumentacja pliku core/Position.cpp	82
7.19.1 Dokumentacja funkcji	82
7.19.1.1 operator<<()	82
7.20 Dokumentacja pliku core/Position.h	83
7.21 Position.h	84
7.22 Dokumentacja pliku core/SaveManager.cpp	84
7.23 Dokumentacja pliku core/SaveManager.h	85
7.24 SaveManager.h	85
7.25 Dokumentacja pliku core/Solver.cpp	86
7.26 Dokumentacja pliku core/Solver.h	87
7.27 Solver.h	87
7.28 Dokumentacja pliku core/StandardSudokuMode.cpp	88
7.29 Dokumentacja pliku core/StandardSudokuMode.h	89
7.30 StandardSudokuMode.h	89
7.31 Dokumentacja pliku core/SudokuGame.cpp	90
7.32 Dokumentacja pliku core/SudokuGame.h	91
7.33 SudokuGame.h	91
7.34 Dokumentacja pliku core/SudokuModeBase.cpp	92
7.35 Dokumentacja pliku core/SudokuModeBase.h	92
7.36 SudokuModeBase.h	93
7.37 Dokumentacja pliku core/Sudokurules.cpp	94
7.38 Dokumentacja pliku core/Sudokurules.h	95
7.39 Sudokurules.h	95
7.40 Dokumentacja pliku main.cpp	96
7.40.1 Dokumentacja funkcji	96
7.40.1.1 main()	96

Indeks

99

Rozdział 1

Struktura katalogów

1.1 Katalogi

core	9
Board.cpp	67
Board.h	67
Cell.cpp	69
Cell.h	69
ConsoleUI.cpp	70
ConsoleUI.h	71
GameMode.cpp	73
GameMode.h	75
Generator.cpp	78
Generator.h	79
LargeSudokuMode.cpp	80
LargeSudokuMode.h	81
Position.cpp	82
Position.h	83
SaveManager.cpp	84
SaveManager.h	85
Solver.cpp	86
Solver.h	87
StandardSudokuMode.cpp	88
StandardSudokuMode.h	89
SudokuGame.cpp	90
SudokuGame.h	91
SudokuModeBase.cpp	92
SudokuModeBase.h	92
Sodokurules.cpp	94
Sodokurules.h	95

Rozdział 2

Indeks hierarchiczny

2.1 Hierarchia klas

Ta lista dziedziczenia posortowana jest z grubsza, choć nie całkowicie, alfabetycznie:

Board	11
Cell	19
ConsoleUI	23
GameMode	24
SudokuModeBase	49
LargeSudokuMode	32
StandardSudokuMode	42
Generator	31
Position	35
SaveManager	39
Solver	41
SudokuGame	45
SudokuRules	60

Rozdział 3

Indeks klas

3.1 Lista klas

Tutaj znajdują się klasy, struktury, unie i interfejsy wraz z ich krótkimi opisami:

Board	Reprezentuje planszę Sudoku 9x9	11
Cell	Reprezentuje pojedyncze pole planszy Sudoku	19
ConsoleUI	Obsługuje menu i interaktywny terminalowy interfejs gry Sudoku	23
GameMode	Abstrakcyjna baza dla trybów gry	24
Generator	Odpowiada za generowanie plansz Sudoku	31
LargeSudokuMode	Rozszerzone Sudoku z blokami 4x4 i planszą 16x16	32
Position	Reprezentuje położenie pola na planszy	35
SaveManager	Zapisuje i odczytuje stan gry z pliku tekstowego	39
Solver	Odpowiada za rozwiązywanie planszy Sudoku	41
StandardSudokuMode	Klasyczne Sudoku z blokami 3x3 i planszą 9x9	42
SudokuGame	Zarządza aktualną rozgrywką Sudoku	45
SudokuModeBase	Wspólna baza dla konkretnych wariantów Sudoku	49
SudokuRules	Odpowiada za sprawdzanie zasad gry Sudoku	60

Rozdział 4

Indeks plików

4.1 Lista plików

Tutaj znajduje się lista wszystkich plików wraz z ich krótkimi opisami:

main.cpp	96
core/Board.cpp	67
core/Board.h	67
core/Cell.cpp	69
core/Cell.h	69
core/ConsoleUI.cpp	70
core/ConsoleUI.h	71
core/GameMode.cpp	73
core/GameMode.h	75
core/Generator.cpp	78
core/Generator.h	79
core/LargeSudokuMode.cpp	80
core/LargeSudokuMode.h	81
core/Position.cpp	82
core/Position.h	83
core/SaveManager.cpp	84
core/SaveManager.h	85
core/Solver.cpp	86
core/Solver.h	87
core/StandardSudokuMode.cpp	88
core/StandardSudokuMode.h	89
core/SudokuGame.cpp	90
core/SudokuGame.h	91
core/SudokuModeBase.cpp	92
core/SudokuModeBase.h	92
core/Sudokurules.cpp	94
core/Sudokurules.h	95

Rozdział 5

Dokumentacja katalogów

5.1 Dokumentacja katalogu core

Pliki

- plik [Board.cpp](#)
- plik [Board.h](#)
- plik [Cell.cpp](#)
- plik [Cell.h](#)
- plik [ConsoleUI.cpp](#)
- plik [ConsoleUI.h](#)
- plik [GameMode.cpp](#)
- plik [GameMode.h](#)
- plik [Generator.cpp](#)
- plik [Generator.h](#)
- plik [LargeSudokuMode.cpp](#)
- plik [LargeSudokuMode.h](#)
- plik [Position.cpp](#)
- plik [Position.h](#)
- plik [SaveManager.cpp](#)
- plik [SaveManager.h](#)
- plik [Solver.cpp](#)
- plik [Solver.h](#)
- plik [StandardSudokuMode.cpp](#)
- plik [StandardSudokuMode.h](#)
- plik [SudokuGame.cpp](#)
- plik [SudokuGame.h](#)
- plik [SudokuModeBase.cpp](#)
- plik [SudokuModeBase.h](#)
- plik [Sudokurules.cpp](#)
- plik [Sudokurules.h](#)

Rozdział 6

Dokumentacja klas

6.1 Dokumentacja klasy Board

Reprezentuje planszę Sudoku 9x9.

```
#include <Board.h>
```

Metody publiczne

- `Board ()`
Tworzy pustą planszę Sudoku.
- `Cell & at (int row, int col)`
Zwraca referencję do pola na podanej pozycji.
- `const Cell & at (int row, int col) const`
Zwraca stałą referencję do pola na podanej pozycji.
- `Cell & operator() (int row, int col)`
Przeciążenie operatora dostępu do pola planszy.
- `const Cell & operator() (int row, int col) const`
Stać wersja operatora dostępu do pola planszy.
- `int getValue (int row, int col) const`
Pobiera wartość z wybranego pola.
- `void setValue (int row, int col, int value)`
Ustawia wartość wybranego pola.
- `bool isEmpty (int row, int col) const`
Sprawdza, czy wybrane pole jest puste.
- `bool isFixed (int row, int col) const`
Sprawdza, czy wybrane pole jest stałe.
- `bool isFull () const`
Sprawdza, czy plansza jest całkowicie wypełniona.
- `void clear ()`
Czyści całą planszę.

Statyczne atrybuty publiczne

- static const int [SIZE](#) = 9
Rozmiar planszy Sudoku.
- static const int [BOX_SIZE](#) = 3
Rozmiar pojedynczego bloku 3x3.

6.1.1 Opis szczegółowy

Reprezentuje planszę Sudoku 9x9.

Klasa przechowuje wszystkie pola planszy oraz udostępnia metody do odczytu i modyfikacji ich zawartości.

6.1.2 Dokumentacja konstruktora i destruktor

6.1.2.1 Board()

```
Board::Board ()
```

Tworzy pustą planszę Sudoku.

6.1.3 Dokumentacja funkcji składowych

6.1.3.1 at() [1/2]

```
Cell & Board::at (
    int row,
    int col)
```

Zwraca referencję do pola na podanej pozycji.

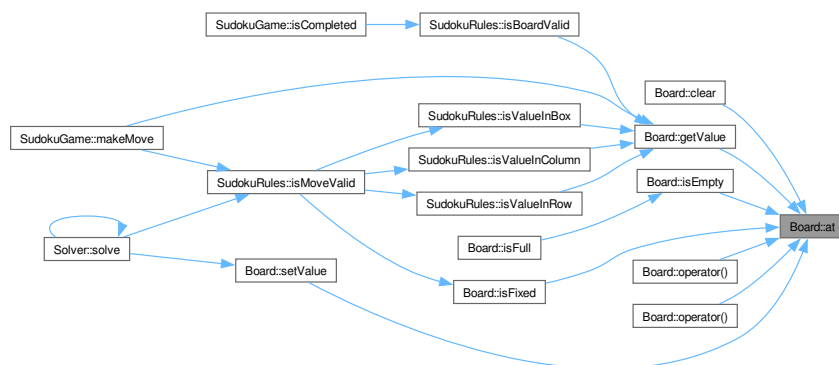
Parametry

<i>row</i>	Numer wiersza.
<i>col</i>	Numer kolumny.

Zwraca

Referencja do obiektu [Cell](#).

Oto graf wywołań tej funkcji:



6.1.3.2 at() [2/2]

```
const Cell & Board::at (  
    int row,  
    int col) const
```

Zwraca stałą referencję do pola na podanej pozycji.

Parametry

<i>row</i>	Numer wiersza.
<i>col</i>	Numer kolumny.

Zwraca

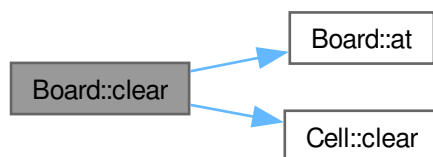
Stała referencja do obiektu [Cell](#).

6.1.3.3 clear()

```
void Board::clear ()
```

Czyści całą planszę.

Wszystkie pola zostają przywrócone do stanu początkowego. Oto graf wywołań dla tej funkcji:



6.1.3.4 getValue()

```
int Board::getValue (  
    int row,  
    int col) const
```

Pobiera wartość z wybranego pola.

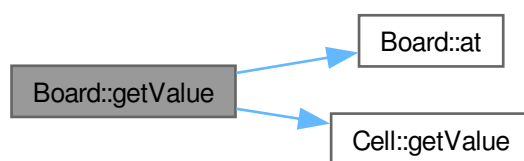
Parametry

<i>row</i>	Numer wiersza.
<i>col</i>	Numer kolumny.

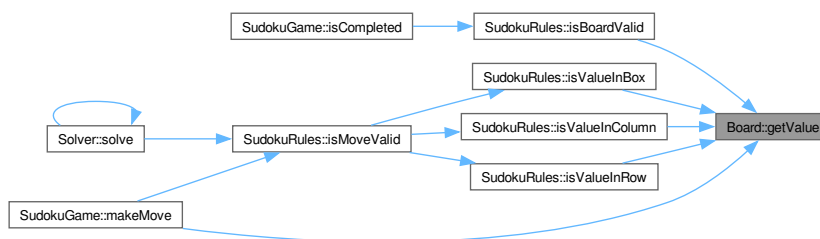
Zwraca

Wartość przechowywana w polu.

Oto graf wywołań dla tej funkcji:



Oto graf wywoływań tej funkcji:

**6.1.3.5 isEmpty()**

```

bool Board::isEmpty (
    int row,
    int col) const
  
```

Sprawdza, czy wybrane pole jest puste.

Parametry

<i>row</i>	Numer wiersza.
------------	----------------

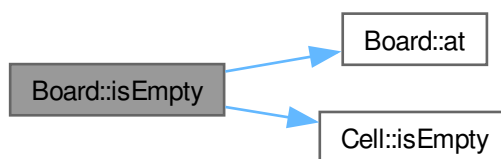
<i>col</i>	Numer kolumny.
------------	----------------

Zwraca

true jeśli pole jest puste.

false jeśli pole zawiera wartość.

Oto graf wywołań dla tej funkcji:



Oto graf wywoływań tej funkcji:

**6.1.3.6 isFixed()**

```
bool Board::isFixed (  
    int row,  
    int col) const
```

Sprawdza, czy wybrane pole jest stałe.

Parametry

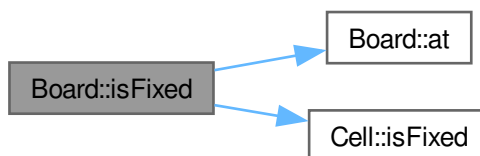
<i>row</i>	Numer wiersza.
<i>col</i>	Numer kolumny.

Zwraca

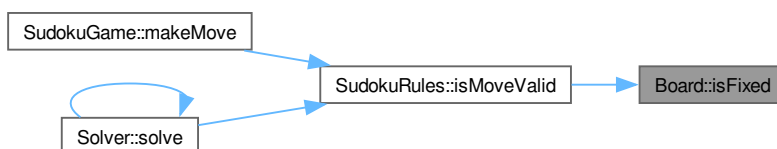
true jeśli pole jest stałe.

false jeśli pole może być edytowane.

Oto graf wywołań dla tej funkcji:



Oto graf wywoływań tej funkcji:

**6.1.3.7 isFull()**

```
bool Board::isFull () const
```

Sprawdza, czy plansza jest całkowicie wypełniona.

Zwraca

true jeśli wszystkie pola zawierają wartości.

false jeśli istnieje przynajmniej jedno puste pole.

Oto graf wywołań dla tej funkcji:



6.1.3.8 operator() [1/2]

```
Cell & Board::operator() (  
    int row,  
    int col)
```

Przeciążenie operatora dostępu do pola planszy.

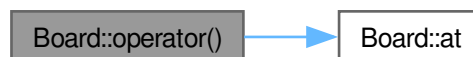
Parametry

<i>row</i>	Numer wiersza.
<i>col</i>	Numer kolumny.

Zwraca

Referencja do obiektu [Cell](#).

Oto graf wywołań dla tej funkcji:



6.1.3.9 operator() [2/2]

```
const Cell & Board::operator() (  
    int row,  
    int col) const
```

Stała wersja operatora dostępu do pola planszy.

Parametry

<i>row</i>	Numer wiersza.
<i>col</i>	Numer kolumny.

Zwraca

Stała referencja do obiektu [Cell](#).

Oto graf wywołań dla tej funkcji:

**6.1.3.10 setValue()**

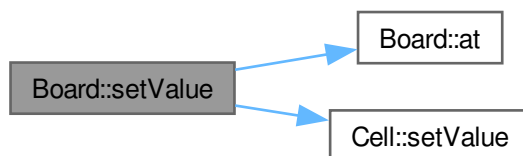
```
void Board::setValue (  
    int row,  
    int col,  
    int value)
```

Ustawia wartość wybranego pola.

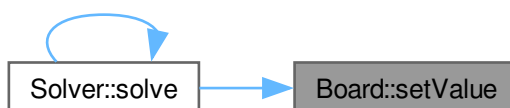
Parametry

<i>row</i>	Numer wiersza.
<i>col</i>	Numer kolumny.
<i>value</i>	Nowa wartość pola.

Oto graf wywołań dla tej funkcji:



Oto graf wywoływań tej funkcji:



6.1.4 Dokumentacja atrybutów składowych

6.1.4.1 BOX_SIZE

```
const int Board::BOX_SIZE = 3 [static]
```

Rozmiar pojedynczego bloku 3x3.

6.1.4.2 SIZE

```
const int Board::SIZE = 9 [static]
```

Rozmiar planszy Sudoku.

Dokumentacja dla tej klasy została wygenerowana z plików:

- [core/Board.h](#)
- [core/Board.cpp](#)

6.2 Dokumentacja klasy Cell

Reprezentuje pojedyncze pole planszy Sudoku.

```
#include <Cell.h>
```

Metody publiczne

- [Cell](#) ()
Tworzy puste, edytowalne pole.
- [Cell](#) (int value, bool fixed)
Tworzy pole z określoną wartością.
- int [getValue](#) () const
Zwraca wartość pola.
- void [setValue](#) (int value)
Ustawia nową wartość pola.
- bool [isFixed](#) () const
Sprawdza, czy pole jest stałe.
- void [setFixed](#) (bool fixed)
Ustawia stan pola.
- bool [isEmpty](#) () const
Sprawdza, czy pole jest puste.
- void [clear](#) ()
Czyści zawartość pola.

6.2.1 Opis szczegółowy

Reprezentuje pojedyncze pole planszy Sudoku.

Klasa przechowuje wartość pola oraz informację, czy pole jest stałym elementem wygenerowanej planszy. Pole stałe nie może być modyfikowane przez gracza.

6.2.2 Dokumentacja konstruktora i destruktor

6.2.2.1 Cell() [1/2]

```
Cell::Cell ()
```

Tworzy puste, edytowalne pole.

6.2.2.2 Cell() [2/2]

```
Cell::Cell (  
    int value,  
    bool fixed)
```

Tworzy pole z określoną wartością.

Parametry

<i>value</i>	Wartość początkowa pola.
<i>fixed</i>	Określa, czy pole jest stałe.

6.2.3 Dokumentacja funkcji składowych

6.2.3.1 clear()

```
void Cell::clear ()
```

Czyści zawartość pola.

Jeśli pole nie jest stałe, jego wartość zostanie ustawiona na 0. Pole stałe pozostaje niezmienione. Oto graf wywołań tej funkcji:



6.2.3.2 getValue()

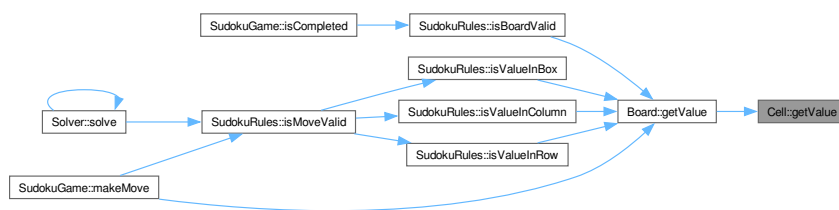
```
int Cell::getValue () const
```

Zwraca wartość pola.

Zwraca

Aktualna wartość pola.

Oto graf wywołań tej funkcji:



6.2.3.3 isEmpty()

```
bool Cell::isEmpty () const
```

Sprawdza, czy pole jest puste.

Pole jest uznawane za puste, gdy jego wartość wynosi 0.

Zwraca

true jeśli pole jest puste.

false jeśli pole zawiera liczbę.

Oto graf wywołań tej funkcji:



6.2.3.4 isFixed()

```
bool Cell::isFixed () const
```

Sprawdza, czy pole jest stałe.

Zwraca

true jeśli pole jest stałe.

false jeśli pole jest edytowalne.

Oto graf wywoływań tej funkcji:



6.2.3.5 setFixed()

```
void Cell::setFixed (
    bool fixed)
```

Ustawia stan pola.

Parametry

<i>fixed</i>	Nowy stan pola.
--------------	-----------------

6.2.3.6 setValue()

```
void Cell::setValue (
    int value)
```

Ustawia nową wartość pola.

Parametry

<i>value</i>	Wartość do zapisania w polu.
--------------	------------------------------

Oto graf wywoływań tej funkcji:



Dokumentacja dla tej klasy została wygenerowana z plików:

- [core/Cell.h](#)
- [core/Cell.cpp](#)

6.3 Dokumentacja klasy ConsoleUI

Obsługuje menu i interaktywny terminalowy interfejs gry Sudoku.

```
#include <ConsoleUI.h>
```

Metody publiczne

- [ConsoleUI \(\)](#)
Tworzy interfejs z kursorem ustawionym w lewym górnym rogu.
- void [run \(\)](#)
Uruchamia program od menu głównego.

6.3.1 Opis szczegółowy

Obsługuje menu i interaktywny terminalowy interfejs gry Sudoku.

Klasa pokazuje praktyczne użycie polimorfizmu: przechowuje aktywną grę jako wskaźnik do [GameMode](#), a rzeczywisty tryb może być planszą 9x9 albo 16x16.

6.3.2 Dokumentacja konstruktora i destruktor

6.3.2.1 ConsoleUI()

```
ConsoleUI::ConsoleUI ()
```

Tworzy interfejs z kursorem ustawionym w lewym górnym rogu.

6.3.3 Dokumentacja funkcji składowych

6.3.3.1 run()

```
void ConsoleUI::run ()
```

Uruchamia program od menu głównego.

Oto graf wywoływań tej funkcji:



Dokumentacja dla tej klasy została wygenerowana z plików:

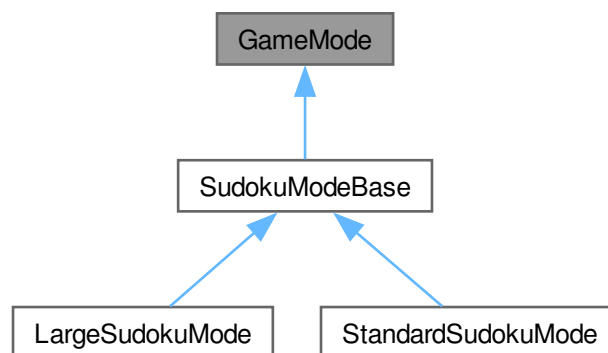
- [core/ConsoleUI.h](#)
- [core/ConsoleUI.cpp](#)

6.4 Dokumentacja klasy GameMode

Abstrakcyjna baza dla trybów gry.

```
#include <GameMode.h>
```

Diagram dziedziczenia dla GameMode



Metody publiczne

- virtual `~GameMode ()`=default
Wirtualny destruktor umożliwia poprawne niszczenie klas pochodnych.
- virtual void `startNew (Difficulty difficulty)`=0
Rozpoczyna nową rozgrywkę.
- virtual std::string `getName ()` const =0
Zwraca nazwę trybu gry.
- virtual `Difficulty getDifficulty ()` const =0
Zwraca poziom trudności aktualnej rozgrywki.
- virtual int `getSize ()` const =0
Zwraca rozmiar planszy.
- virtual int `getBoxSize ()` const =0
Zwraca rozmiar bloku.
- virtual int `getMaxValue ()` const =0
Zwraca największą dopuszczalną wartość.
- virtual int `getValue (int row, int col)` const =0
Pobiera wartość pola.
- virtual bool `isFixed (int row, int col)` const =0
Sprawdza, czy pole jest polem startowym.
- virtual bool `makeMove (int row, int col, int value)`=0
Wpisuje wartość w pole.
- virtual bool `clearCell (int row, int col)`=0
Czyści pole gracza.
- virtual bool `isCompleted ()` const =0
Sprawdza, czy plansza jest poprawnie ukończona.
- virtual bool `loadState (const int *values, const int *fixed, Difficulty difficulty)`=0
Wczytuje stan planszy z danych liczbowych.

6.4.1 Opis szczegółowy

Abstrakcyjna baza dla trybów gry.

Klasa pokazuje polimorfizm: interfejs użytkownika pracuje na wskaźniku do `GameMode`, a konkretne zachowanie zależy od rzeczywistej klasy pochodnej.

6.4.2 Dokumentacja konstruktora i destruktor

6.4.2.1 `~GameMode()`

```
virtual GameMode::~~GameMode () [virtual], [default]
```

Wirtualny destruktor umożliwia poprawne niszczenie klas pochodnych.

6.4.3 Dokumentacja funkcji składowych

6.4.3.1 clearCell()

```
virtual bool GameMode::clearCell (  
    int row,  
    int col) [pure virtual]
```

Czyści pole gracza.

Parametry

<i>row</i>	Numer wiersza.
<i>col</i>	Numer kolumny.

Zwraca

true jeśli pole zostało wyczyszczone.

Implementowany w [SudokuModeBase](#).

6.4.3.2 getBoxSize()

```
virtual int GameMode::getBoxSize () const [pure virtual]
```

Zwraca rozmiar bloku.

Zwraca

Rozmiar jednego boku bloku.

Implementowany w [SudokuModeBase](#).

Oto graf wywoływań tej funkcji:



6.4.3.3 getDifficulty()

```
virtual Difficulty GameMode::getDifficulty () const [pure virtual]
```

Zwraca poziom trudności aktualnej rozgrywki.

Zwraca

Poziom trudności.

Implementowany w [SudokuModeBase](#).

Oto graf wywoływań tej funkcji:



6.4.3.4 getMaxValue()

```
virtual int GameMode::getMaxValue () const [pure virtual]
```

Zwraca największą dopuszczalną wartość.

Zwraca

Maksymalna wartość pola.

Implementowany w [SudokuModeBase](#).

6.4.3.5 getName()

```
virtual std::string GameMode::getName () const [pure virtual]
```

Zwraca nazwę trybu gry.

Zwraca

Nazwa trybu.

Implementowany w [LargeSudokuMode](#) i [StandardSudokuMode](#).

6.4.3.6 getSize()

```
virtual int GameMode::getSize () const [pure virtual]
```

Zwraca rozmiar planszy.

Zwraca

Liczba wierszy i kolumn planszy.

Implementowany w [SudokuModeBase](#).

Oto graf wywołań tej funkcji:



6.4.3.7 getValue()

```
virtual int GameMode::getValue (  
    int row,  
    int col) const [pure virtual]
```

Pobiera wartość pola.

Parametry

<i>row</i>	Numer wiersza.
<i>col</i>	Numer kolumny.

Zwraca

Wartość pola.

Implementowany w [SudokuModeBase](#).

Oto graf wywołań tej funkcji:



6.4.3.8 isCompleted()

```
virtual bool GameMode::isCompleted () const [pure virtual]
```

Sprawdza, czy plansza jest poprawnie ukończona.

Zwraca

true jeśli rozgrywka jest zakończona.

Implementowany w [SudokuModeBase](#).

6.4.3.9 isFixed()

```
virtual bool GameMode::isFixed (  
    int row,  
    int col) const [pure virtual]
```

Sprawdza, czy pole jest polem startowym.

Parametry

<i>row</i>	Numer wiersza.
<i>col</i>	Numer kolumny.

Zwraca

true jeśli pole jest stałe.

Implementowany w [SudokuModeBase](#).

Oto graf wywoływań tej funkcji:



6.4.3.10 loadState()

```
virtual bool GameMode::loadState (
    const int * values,
    const int * fixed,
    Difficulty difficulty) [pure virtual]
```

Wczytuje stan planszy z danych liczbowych.

Parametry

<i>values</i>	Wartości pól.
<i>fixed</i>	Informacja, które pola są stałe.
<i>difficulty</i>	Poziom trudności zapisu.

Zwraca

true jeśli dane pasują do trybu gry.

Implementowany w [SudokuModeBase](#).

6.4.3.11 makeMove()

```
virtual bool GameMode::makeMove (
    int row,
    int col,
    int value) [pure virtual]
```

Wpisuje wartość w pole.

Parametry

<i>row</i>	Numer wiersza.
<i>col</i>	Numer kolumny.
<i>value</i>	Wpisywana wartość.

Zwraca

true jeśli ruch został przyjęty.

Implementowany w [SudokuModeBase](#).

6.4.3.12 startNew()

```
virtual void GameMode::startNew (
    Difficulty difficulty) [pure virtual]
```

Rozpoczyna nową rozgrywkę.

Parametry

<i>difficulty</i>	Wybrany poziom trudności.
-------------------	---------------------------

Implementowany w [SudokuModeBase](#).

Dokumentacja dla tej klasy została wygenerowana z pliku:

- [core/GameMode.h](#)

6.5 Dokumentacja klasy Generator

Odpowiada za generowanie plansz Sudoku.

```
#include <Generator.h>
```

Metody publiczne

- [Board generatePuzzle](#) (int emptyCells)
Generuje planszę startową Sudoku.

6.5.1 Opis szczegółowy

Odpowiada za generowanie plansz Sudoku.

Klasa tworzy pełne poprawne plansze Sudoku, a następnie usuwa część pól, aby powstała zagadka.

6.5.2 Dokumentacja funkcji składowych

6.5.2.1 generatePuzzle()

```
Board Generator::generatePuzzle (
    int emptyCells)
```

Generuje planszę startową Sudoku.

Parametry

<i>emptyCells</i>	Liczba pól, które mają zostać usunięte.
-------------------	---

Zwraca

Wygenerowana plansza Sudoku.

Oto graf wywoływań tej funkcji:



Dokumentacja dla tej klasy została wygenerowana z plików:

- [core/Generator.h](#)
- [core/Generator.cpp](#)

6.6 Dokumentacja klasy LargeSudokuMode

Rozszerzone Sudoku z blokami 4x4 i planszą 16x16.

```
#include <LargeSudokuMode.h>
```

Diagram dziedziczenia dla LargeSudokuMode

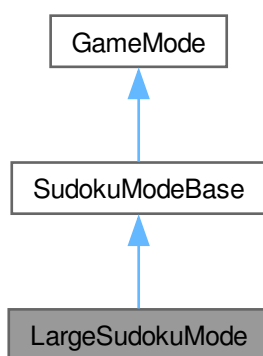
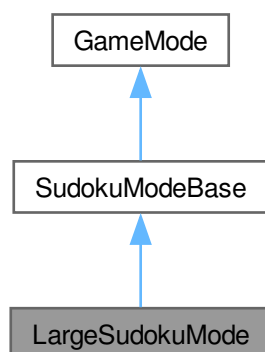


Diagram współpracy dla LargeSudokuMode:



Metody publiczne

- [LargeSudokuMode](#) ()
Tworzy rozszerzony tryb Sudoku.
- `std::string` [getName](#) () const override
Zwraca nazwę trybu gry.

Metody publiczne dziedziczone z [SudokuModeBase](#)

- [SudokuModeBase](#) (int [boxSize](#))
Tworzy bazę trybu Sudoku.
- void [startNew](#) ([Difficulty](#) [difficulty](#)) override
Rozpoczyna nową rozgrywkę.
- [Difficulty](#) [getDifficulty](#) () const override
Zwraca poziom trudności aktualnej rozgrywki.
- int [getSize](#) () const override
Zwraca rozmiar planszy.
- int [getBoxSize](#) () const override
Zwraca rozmiar bloku.
- int [getMaxValue](#) () const override
Zwraca największą dopuszczalną wartość.
- int [getValue](#) (int row, int col) const override
Pobiera wartość pola.
- bool [isFixed](#) (int row, int col) const override
Sprawdza, czy pole jest polem startowym.
- bool [makeMove](#) (int row, int col, int value) override
Wpisuje wartość w pole.
- bool [clearCell](#) (int row, int col) override
Czyści pole gracza.
- bool [isCompleted](#) () const override
Sprawdza, czy plansza jest poprawnie ukończona.
- bool [loadState](#) (const int *[values](#), const int *[fixed](#), [Difficulty](#) [difficulty](#)) override
Wczytuje stan planszy z danych liczbowych.

Metody publiczne dziedziczone z `GameMode`

- virtual `~GameMode()`=default

Wirtualny destruktor umożliwia poprawne niszczenie klas pochodnych.

Dodatkowe dziedziczone składowe

Metody chronione dziedziczone z `SudokuModeBase`

- int `index` (int row, int col) const
Zwraca indeks pola w jednowymiarowej tablicy.
- bool `isInside` (int row, int col) const
Sprawdza, czy współrzędne mieszczą się na planszy.
- int `emptyCellsForDifficulty` (Difficulty difficulty) const
Ustala liczbę pól do usunięcia dla poziomu trudności.
- std::vector< int > `generateSolvedValues` () const
Generuje pełną, poprawną planszę.
- bool `canPlaceValue` (int row, int col, int value) const
Sprawdza, czy wartość nie koliduje z zasadami Sudoku.

Atrybuty chronione dziedziczone z `SudokuModeBase`

- int `boxSize`
- int `size`
- Difficulty `difficulty`
- std::vector< int > `values`
- std::vector< int > `fixedFields`

6.6.1 Opis szczegółowy

Rozszerzone Sudoku z blokami 4x4 i planszą 16x16.

6.6.2 Dokumentacja konstruktora i destruktor

6.6.2.1 `LargeSudokuMode()`

```
LargeSudokuMode::LargeSudokuMode ()
```

Tworzy rozszerzony tryb Sudoku.

Oto graf wywołań dla tej funkcji:



6.6.3 Dokumentacja funkcji składowych

6.6.3.1 getName()

```
std::string LargeSudokuMode::getName () const [override], [virtual]
```

Zwraca nazwę trybu gry.

Zwraca

Nazwa trybu.

Implementuje [GameMode](#).

Dokumentacja dla tej klasy została wygenerowana z plików:

- [core/LargeSudokuMode.h](#)
- [core/LargeSudokuMode.cpp](#)

6.7 Dokumentacja klasy Position

Reprezentuje położenie pola na planszy.

```
#include <Position.h>
```

Metody publiczne

- [Position](#) (int row=0, int col=0)
Tworzy pozycję na planszy.
- int [getRow](#) () const
Zwraca numer wiersza.
- int [getCol](#) () const
Zwraca numer kolumny.
- void [set](#) (int newRow, int newCol)
Ustawia nową pozycję.
- bool [operator==](#) (const [Position](#) &other) const
Porównuje dwie pozycje.
- bool [operator!=](#) (const [Position](#) &other) const
Sprawdza, czy pozycje są różne.

Przyjaciele

- std::ostream & [operator<<](#) (std::ostream &stream, const [Position](#) &position)
Wypisuje pozycję w czytelnej formie.

6.7.1 Opis szczegółowy

Reprezentuje położenie pola na planszy.

Klasa jest małym przykładem przeciążania operatorów. Operator porównania zdefiniowany jest jako metoda składowa, a operator wypisywania jako funkcja zaprzyjaźniona, ponieważ potrzebuje bezpośredniego dostępu do pól klasy.

6.7.2 Dokumentacja konstruktora i destruktora

6.7.2.1 Position()

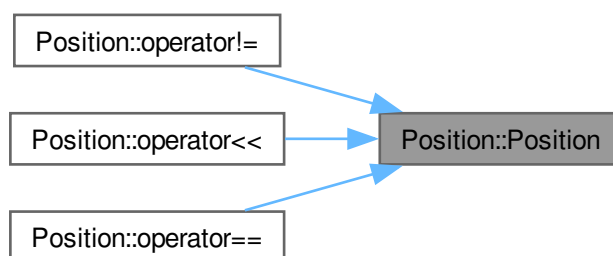
```
Position::Position (
    int row = 0,
    int col = 0)
```

Tworzy pozycję na planszy.

Parametry

<i>row</i>	Numer wiersza liczony od zera.
<i>col</i>	Numer kolumny liczony od zera.

Oto graf wywoływań tej funkcji:



6.7.3 Dokumentacja funkcji składowych

6.7.3.1 getCol()

```
int Position::getCol () const
```

Zwraca numer kolumny.

Zwraca

Numer kolumny liczony od zera.

6.7.3.2 getRow()

```
int Position::getRow () const
```

Zwraca numer wiersza.

Zwraca

Numer wiersza liczony od zera.

6.7.3.3 operator!=(())

```
bool Position::operator!= (
    const Position & other) const
```

Sprawdza, czy pozycje są różne.

Parametry

<i>other</i>	Druga pozycja.
--------------	----------------

Zwraca

true jeśli pozycje wskazują różne pola.

Oto graf wywołań dla tej funkcji:



6.7.3.4 operator==(())

```
bool Position::operator== (
    const Position & other) const
```

Porównuje dwie pozycje.

Parametry

<i>other</i>	Druga pozycja.
--------------	----------------

Zwraca

true jeśli obie pozycje wskazują to samo pole.

Oto graf wywołań dla tej funkcji:

**6.7.3.5 set()**

```
void Position::set (
    int newRow,
    int newCol)
```

Ustawia nową pozycję.

Parametry

<i>newRow</i>	Nowy numer wiersza.
<i>newCol</i>	Nowy numer kolumny.

6.7.4 Dokumentacja przyjaciół i powiązanych symboli**6.7.4.1 operator<<**

```
std::ostream & operator<< (
    std::ostream & stream,
    const Position & position) [friend]
```

Wypisuje pozycję w czytelnej formie.

Funkcja jest zaprzyjaźniona z klasą [Position](#), dlatego może odczytać prywatne pola row i col bez używania getterów.

Parametry

<i>stream</i>	Strumień wyjściowy.
<i>position</i>	Pozycja do wypisania.

Zwraca

Referencja do strumienia wyjściowego.

Dokumentacja dla tej klasy została wygenerowana z plików:

- [core/Position.h](#)
- [core/Position.cpp](#)

6.8 Dokumentacja klasy SaveManager

Zapisuje i odczytuje stan gry z pliku tekstowego.

```
#include <SaveManager.h>
```

Statyczne metody publiczne

- static bool `save` (const `GameMode` &game, const std::string &fileName=`DEFAULT_SAVE_FILE`)
Zapisuje aktualny tryb gry do pliku.
- static std::unique_ptr< `GameMode` > `load` (const std::string &fileName=`DEFAULT_SAVE_FILE`)
Wczytuje tryb gry z pliku.

Statyczne atrybuty publiczne

- static const char * `DEFAULT_SAVE_FILE` = "sudoku_save.txt"
Domyślna ścieżka pliku zapisu.

6.8.1 Opis szczegółowy

Zapisuje i odczytuje stan gry z pliku tekstowego.

6.8.2 Dokumentacja funkcji składowych

6.8.2.1 load()

```
std::unique_ptr< GameMode > SaveManager::load (  
    const std::string & fileName = DEFAULT_SAVE_FILE) [static]
```

Wczytuje tryb gry z pliku.

Parametry

<code>fileName</code>	Nazwa pliku zapisu.
-----------------------	---------------------

Zwraca

Wskaźnik do odtworzonego trybu gry albo nullptr przy błędzie.

Oto graf wywołań dla tej funkcji:



6.8.2.2 save()

```
bool SaveManager::save (
    const GameMode & game,
    const std::string & fileName = DEFAULT_SAVE_FILE) [static]
```

Zapisuje aktualny tryb gry do pliku.

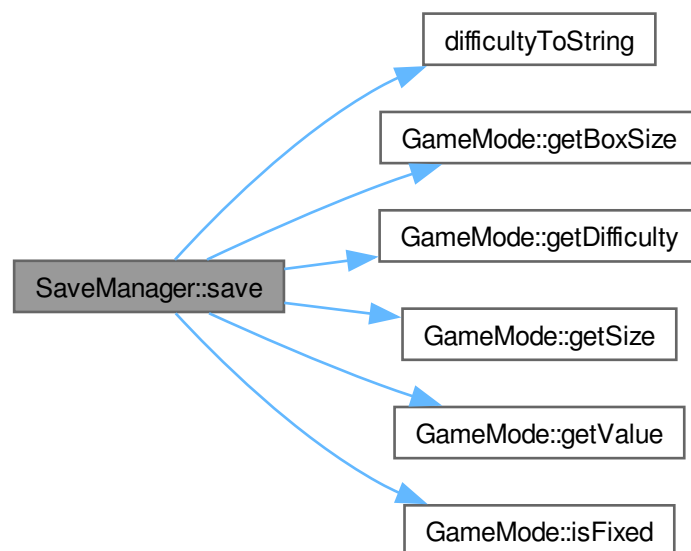
Parametry

<i>game</i>	Tryb gry do zapisania.
<i>fileName</i>	Nazwa pliku zapisu.

Zwraca

true jeśli zapis się udał.

Oto graf wywołań dla tej funkcji:



6.8.3 Dokumentacja atrybutów składowych

6.8.3.1 DEFAULT_SAVE_FILE

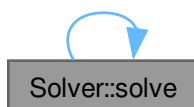
```
const char * SaveManager::DEFAULT_SAVE_FILE = "sudoku_save.txt" [static]
```

Domyślna ścieżka pliku zapisu.

Dokumentacja dla tej klasy została wygenerowana z plików:

- [core/SaveManager.h](#)
- [core/SaveManager.cpp](#)

Oto graf wywoływań tej funkcji:



Dokumentacja dla tej klasy została wygenerowana z plików:

- [core/Solver.h](#)
- [core/Solver.cpp](#)

6.10 Dokumentacja klasy StandardSudokuMode

Klasyczne Sudoku z blokami 3x3 i planszą 9x9.

```
#include <StandardSudokuMode.h>
```

Diagram dziedziczenia dla StandardSudokuMode

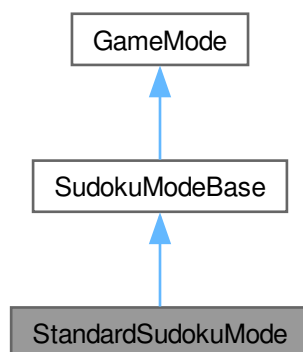
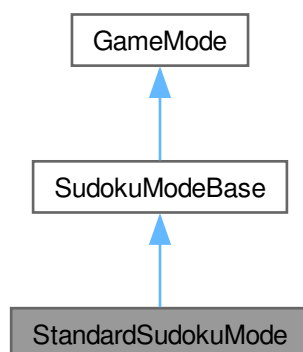


Diagram współpracy dla StandardSudokuMode:



Metody publiczne

- [StandardSudokuMode](#) ()
Tworzy klasyczny tryb Sudoku.
- `std::string` [getName](#) () const override
Zwraca nazwę trybu gry.

Metody publiczne dziedziczone z [SudokuModeBase](#)

- [SudokuModeBase](#) (int [boxSize](#))
Tworzy bazę trybu Sudoku.
- void [startNew](#) ([Difficulty](#) [difficulty](#)) override
Rozpoczyna nową rozgrywkę.
- [Difficulty](#) [getDifficulty](#) () const override
Zwraca poziom trudności aktualnej rozgrywki.
- int [getSize](#) () const override
Zwraca rozmiar planszy.
- int [getBoxSize](#) () const override
Zwraca rozmiar bloku.
- int [getMaxValue](#) () const override
Zwraca największą dopuszczalną wartość.
- int [getValue](#) (int row, int col) const override
Pobiera wartość pola.
- bool [isFixed](#) (int row, int col) const override
Sprawdza, czy pole jest polem startowym.
- bool [makeMove](#) (int row, int col, int value) override
Wpisuje wartość w pole.
- bool [clearCell](#) (int row, int col) override
Czyści pole gracza.
- bool [isCompleted](#) () const override
Sprawdza, czy plansza jest poprawnie ukończona.
- bool [loadState](#) (const int *[values](#), const int *[fixed](#), [Difficulty](#) [difficulty](#)) override
Wczytuje stan planszy z danych liczbowych.

Metody publiczne dziedziczone z `GameMode`

- virtual `~GameMode()`=default

Wirtualny destruktor umożliwia poprawne niszczenie klas pochodnych.

Dodatkowe dziedziczone składowe

Metody chronione dziedziczone z `SudokuModeBase`

- int `index` (int row, int col) const
Zwraca indeks pola w jednowymiarowej tablicy.
- bool `isInside` (int row, int col) const
Sprawdza, czy współrzędne mieszczą się na planszy.
- int `emptyCellsForDifficulty` (Difficulty difficulty) const
Ustala liczbę pól do usunięcia dla poziomu trudności.
- std::vector< int > `generateSolvedValues` () const
Generuje pełną, poprawną planszę.
- bool `canPlaceValue` (int row, int col, int value) const
Sprawdza, czy wartość nie koliduje z zasadami Sudoku.

Atrybuty chronione dziedziczone z `SudokuModeBase`

- int `boxSize`
- int `size`
- Difficulty `difficulty`
- std::vector< int > `values`
- std::vector< int > `fixedFields`

6.10.1 Opis szczegółowy

Klasyczne Sudoku z blokami 3x3 i planszą 9x9.

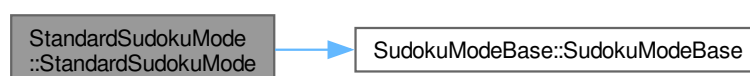
6.10.2 Dokumentacja konstruktora i destruktor

6.10.2.1 `StandardSudokuMode()`

```
StandardSudokuMode::StandardSudokuMode ()
```

Tworzy klasyczny tryb Sudoku.

Oto graf wywołań dla tej funkcji:



6.10.3 Dokumentacja funkcji składowych

6.10.3.1 getName()

```
std::string StandardSudokuMode::getName () const [override], [virtual]
```

Zwraca nazwę trybu gry.

Zwraca

Nazwa trybu.

Implementuje [GameMode](#).

Dokumentacja dla tej klasy została wygenerowana z plików:

- [core/StandardSudokuMode.h](#)
- [core/StandardSudokuMode.cpp](#)

6.11 Dokumentacja klasy SudokuGame

Zarządza aktualną rozgrywką Sudoku.

```
#include <SudokuGame.h>
```

Metody publiczne

- [SudokuGame](#) ()
Tworzy pustą grę Sudoku.
- void [newGame](#) (int emptyCells)
Rozpoczyna nową grę.
- bool [makeMove](#) (int row, int col, int value)
Próbuje wpisać wartość w wybrane pole planszy.
- bool [clearCell](#) (int row, int col)
Czyści wybrane pole planszy.
- bool [isCompleted](#) () const
Sprawdza, czy gra została ukończona.
- const [Board](#) & [getBoard](#) () const
Zwraca aktualną planszę do odczytu.

6.11.1 Opis szczegółowy

Zarządza aktualną rozgrywką Sudoku.

Klasa przechowuje aktualną planszę gry oraz udostępnia metody do rozpoczęcia nowej gry, wykonywania ruchów i sprawdzania stanu rozgrywki.

6.11.2 Dokumentacja konstruktora i destruktora

6.11.2.1 SudokuGame()

```
SudokuGame::SudokuGame ()
```

Tworzy pustą grę Sudoku.

6.11.3 Dokumentacja funkcji składowych

6.11.3.1 clearCell()

```
bool SudokuGame::clearCell (
    int row,
    int col)
```

Czyści wybrane pole planszy.

Parametry

<i>row</i>	Numer wiersza.
<i>col</i>	Numer kolumny.

Zwraca

true jeśli pole zostało wyczyszczone.

false jeśli pole nie może być wyczyszczone.

6.11.3.2 getBoard()

```
const Board & SudokuGame::getBoard () const
```

Zwraca aktualną planszę do odczytu.

Zwraca

Stała referencja do planszy.

6.11.3.3 isCompleted()

```
bool SudokuGame::isCompleted () const
```

Sprawdza, czy gra została ukończona.

Gra jest ukończona, gdy plansza jest pełna i wszystkie wpisane wartości spełniają zasady Sudoku.

Zwraca

true jeśli plansza jest pełna i poprawna.

false jeśli plansza zawiera puste pola lub błędy.

Oto graf wywołań dla tej funkcji:



6.11.3.4 makeMove()

```
bool SudokuGame::makeMove (  
    int row,  
    int col,  
    int value)
```

Próbuje wpisać wartość w wybrane pole planszy.

Metoda nie pozwala modyfikować pól stałych. Jeśli pole zawierało wcześniej wartość wpisaną przez gracza, zostaje ona tymczasowo usunięta na czas sprawdzania poprawności nowego ruchu. Gdy nowy ruch jest niepoprawny, poprzednia wartość zostaje przywrócona.

Parametry

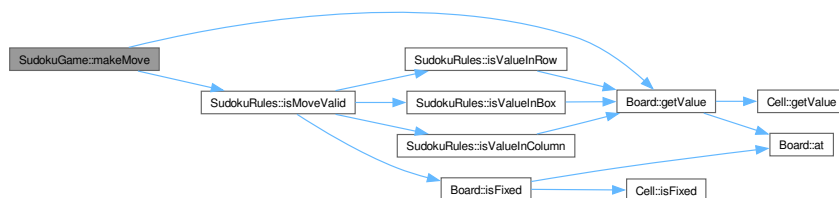
<i>row</i>	Numer wiersza.
<i>col</i>	Numer kolumny.
<i>value</i>	Wartość do wpisania.

Zwraca

true jeśli wartość została wpisana.

false jeśli pole jest stałe lub ruch łamie zasady Sudoku.

Oto graf wywołań dla tej funkcji:

**6.11.3.5 newGame()**

```
void SudokuGame::newGame (
    int emptyCells)
```

Rozpoczyna nową grę.

Parametry

<i>emptyCells</i>	Liczba pól, które mają być puste.
-------------------	-----------------------------------

Oto graf wywołań dla tej funkcji:



Dokumentacja dla tej klasy została wygenerowana z plików:

- [core/SudokuGame.h](#)
- [core/SudokuGame.cpp](#)

6.12 Dokumentacja klasy SudokuModeBase

Wspólna baza dla konkretnych wariantów Sudoku.

```
#include <SudokuModeBase.h>
```

Diagram dziedziczenia dla SudokuModeBase

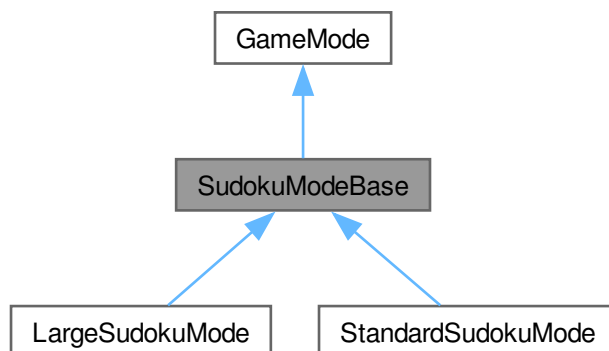
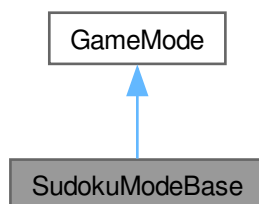


Diagram współpracy dla SudokuModeBase:



Metody publiczne

- `SudokuModeBase` (int `boxSize`)
Tworzy bazę trybu Sudoku.
- void `startNew` (`Difficulty` `difficulty`) override
Rozpoczyna nową rozgrywkę.
- `Difficulty` `getDifficulty` () const override
Zwraca poziom trudności aktualnej rozgrywki.
- int `getSize` () const override
Zwraca rozmiar planszy.

- int `getBoxSize` () const override
Zwraca rozmiar bloku.
- int `getMaxValue` () const override
Zwraca największą dopuszczalną wartość.
- int `getValue` (int row, int col) const override
Pobiera wartość pola.
- bool `isFixed` (int row, int col) const override
Sprawdza, czy pole jest polem startowym.
- bool `makeMove` (int row, int col, int value) override
Wpisuje wartość w pole.
- bool `clearCell` (int row, int col) override
Czyści pole gracza.
- bool `isCompleted` () const override
Sprawdza, czy plansza jest poprawnie ukończona.
- bool `loadState` (const int *values, const int *fixed, Difficulty difficulty) override
Wczytuje stan planszy z danych liczbowych.

Metody publiczne dziedziczone z `GameMode`

- virtual `~GameMode` ()=default
Wirtualny destruktor umożliwia poprawne niszczenie klas pochodnych.
- virtual std::string `getName` () const =0
Zwraca nazwę trybu gry.

Metody chronione

- int `index` (int row, int col) const
Zwraca indeks pola w jednowymiarowej tablicy.
- bool `isInside` (int row, int col) const
Sprawdza, czy współrzędne mieszczą się na planszy.
- int `emptyCellsForDifficulty` (Difficulty difficulty) const
Ustala liczbę pól do usunięcia dla poziomu trudności.
- std::vector< int > `generateSolvedValues` () const
Generuje pełną, poprawną planszę.
- bool `canPlaceValue` (int row, int col, int value) const
Sprawdza, czy wartość nie koliduje z zasadami Sudoku.

Atrybuty chronione

- int `boxSize`
- int `size`
- Difficulty `difficulty`
- std::vector< int > `values`
- std::vector< int > `fixedFields`

6.12.1 Opis szczegółowy

Wspólna baza dla konkretnych wariantów Sudoku.

Klasa implementuje większość logiki gry niezależnej od rozmiaru planszy. Klasy pochodne podają tylko rozmiar bloku, np. 3 dla planszy 9x9 albo 4 dla planszy 16x16.

6.12.2 Dokumentacja konstruktora i destruktoru

6.12.2.1 SudokuModeBase()

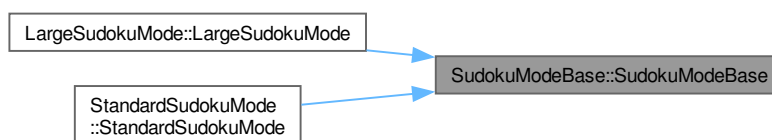
```
SudokuModeBase::SudokuModeBase (  
    int boxSize) [explicit]
```

Tworzy bazę trybu Sudoku.

Parametry

<i>boxSize</i>	Rozmiar boku jednego bloku.
----------------	-----------------------------

Oto graf wywoływań tej funkcji:



6.12.3 Dokumentacja funkcji składowych

6.12.3.1 canPlaceValue()

```
bool SudokuModeBase::canPlaceValue (  
    int row,  
    int col,  
    int value) const [protected]
```

Sprawdza, czy wartość nie koliduje z zasadami Sudoku.

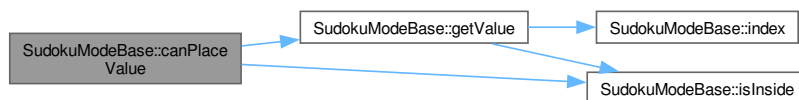
Parametry

<i>row</i>	Numer wiersza.
<i>col</i>	Numer kolumny.
<i>value</i>	Sprawdzana wartość.

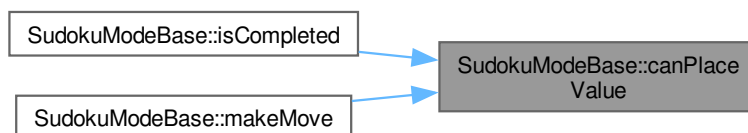
Zwraca

true jeśli wartość można wpisać.

Oto graf wywołań dla tej funkcji:



Oto graf wywoływań tej funkcji:

**6.12.3.2 clearCell()**

```

bool SudokuModeBase::clearCell (
    int row,
    int col) [override], [virtual]
  
```

Czyści pole gracza.

Parametry

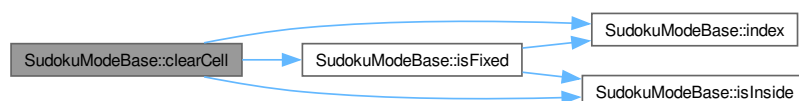
<i>row</i>	Numer wiersza.
<i>col</i>	Numer kolumny.

Zwraca

true jeśli pole zostało wyczyszczone.

Implementuje [GameMode](#).

Oto graf wywołań dla tej funkcji:



6.12.3.3 emptyCellsForDifficulty()

```
int SudokuModeBase::emptyCellsForDifficulty (
    Difficulty difficulty) const [protected]
```

Ustala liczbę pól do usunięcia dla poziomu trudności.

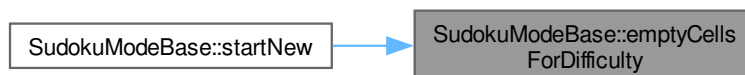
Parametry

<i>difficulty</i>	Wybrany poziom trudności.
-------------------	---------------------------

Zwraca

Liczba pól pustych na starcie.

Oto graf wywołań tej funkcji:



6.12.3.4 generateSolvedValues()

```
std::vector< int > SudokuModeBase::generateSolvedValues () const [protected]
```

Generuje pełną, poprawną planszę.

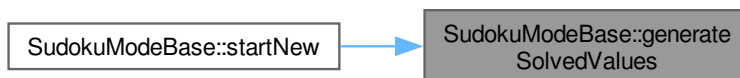
Zwraca

Wektor wartości rozwiązanej planszy.

Oto graf wywołań dla tej funkcji:



Oto graf wywoływań tej funkcji:



6.12.3.5 getBoxSize()

```
int SudokuModeBase::getBoxSize () const [override], [virtual]
```

Zwraca rozmiar bloku.

Zwraca

Rozmiar jednego boku bloku.

Implementuje [GameMode](#).

6.12.3.6 getDifficulty()

```
Difficulty SudokuModeBase::getDifficulty () const [override], [virtual]
```

Zwraca poziom trudności aktualnej rozgrywki.

Zwraca

Poziom trudności.

Implementuje [GameMode](#).

6.12.3.7 getMaxValue()

```
int SudokuModeBase::getMaxValue () const [override], [virtual]
```

Zwraca największą dopuszczalną wartość.

Zwraca

Maksymalna wartość pola.

Implementuje [GameMode](#).

6.12.3.8 getSize()

```
int SudokuModeBase::getSize () const [override], [virtual]
```

Zwraca rozmiar planszy.

Zwraca

Liczba wierszy i kolumn planszy.

Implementuje [GameMode](#).

6.12.3.9 getValue()

```
int SudokuModeBase::getValue (
    int row,
    int col) const [override], [virtual]
```

Pobiera wartość pola.

Parametry

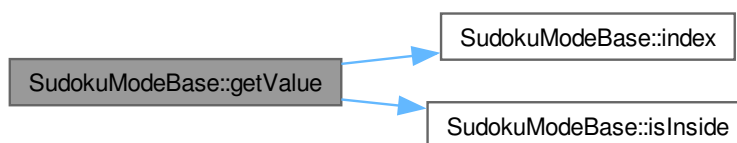
<i>row</i>	Numer wiersza.
<i>col</i>	Numer kolumny.

Zwraca

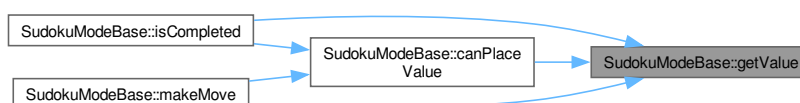
Wartość pola.

Implementuje [GameMode](#).

Oto graf wywołań dla tej funkcji:



Oto graf wywoływań tej funkcji:



6.12.3.10 index()

```
int SudokuModeBase::index (
    int row,
    int col) const [protected]
```

Zwraca indeks pola w jednowymiarowej tablicy.

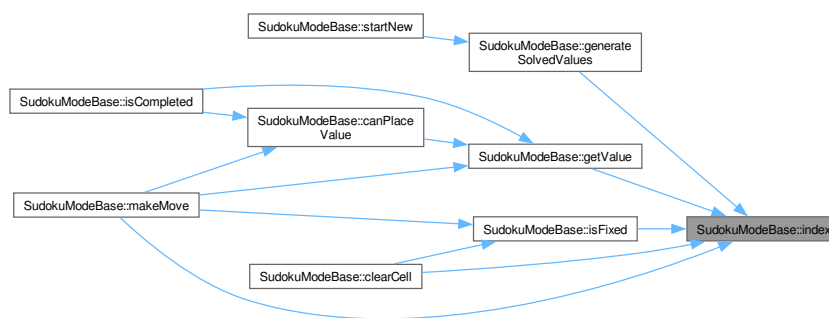
Parametry

<i>row</i>	Numer wiersza.
<i>col</i>	Numer kolumny.

Zwraca

Indeks pola.

Oto graf wywołań tej funkcji:



6.12.3.11 isCompleted()

```
bool SudokuModeBase::isCompleted () const [override], [virtual]
```

Sprawdza, czy plansza jest poprawnie ukończona.

Zwraca

true jeśli rozgrywka jest zakończona.

Implementuje [GameMode](#).

Oto graf wywołań dla tej funkcji:



6.12.3.12 isFixed()

```
bool SudokuModeBase::isFixed (
    int row,
    int col) const [override], [virtual]
```

Sprawdza, czy pole jest polem startowym.

Parametry

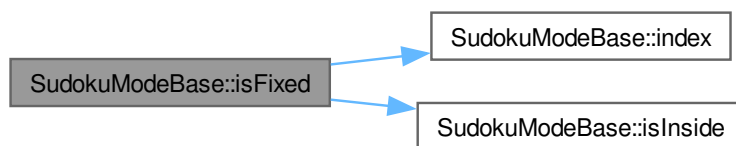
<i>row</i>	Numer wiersza.
<i>col</i>	Numer kolumny.

Zwraca

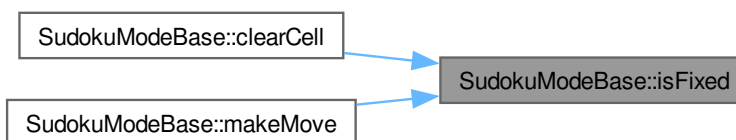
true jeśli pole jest stałe.

Implementuje [GameMode](#).

Oto graf wywołań dla tej funkcji:



Oto graf wywoływań tej funkcji:



6.12.3.13 isInside()

```
bool SudokuModeBase::isInside (
    int row,
    int col) const [protected]
```

Sprawdza, czy współrzędne mieszczą się na planszy.

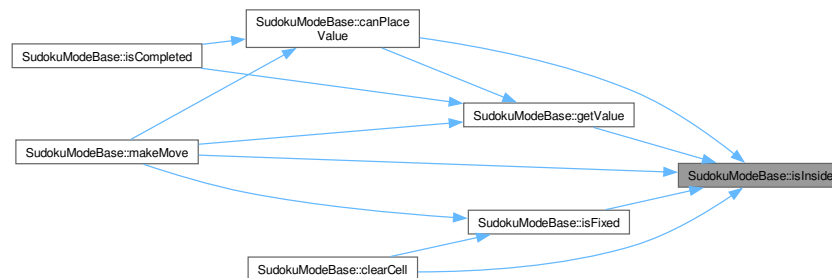
Parametry

<i>row</i>	Numer wiersza.
<i>col</i>	Numer kolumny.

Zwraca

true jeśli współrzędne są poprawne.

Oto graf wywołań tej funkcji:



6.12.3.14 loadState()

```
bool SudokuModeBase::loadState (
    const int * values,
    const int * fixed,
    Difficulty difficulty) [override], [virtual]
```

Wczytuje stan planszy z danych liczbowych.

Parametry

<i>values</i>	Wartości pól.
<i>fixed</i>	Informacja, które pola są stałe.
<i>difficulty</i>	Poziom trudności zapisu.

Zwraca

true jeśli dane pasują do trybu gry.

Implementuje [GameMode](#).

6.12.3.15 makeMove()

```
bool SudokuModeBase::makeMove (
    int row,
    int col,
    int value) [override], [virtual]
```

Wpisuje wartość w pole.

Parametry

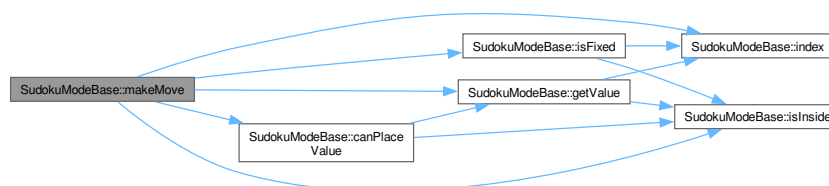
<i>row</i>	Numer wiersza.
<i>col</i>	Numer kolumny.
<i>value</i>	Wpisywana wartość.

Zwraca

true jeśli ruch został przyjęty.

Implementuje [GameMode](#).

Oto graf wywołań dla tej funkcji:



6.12.3.16 startNew()

```
void SudokuModeBase::startNew (
    Difficulty difficulty) [override], [virtual]
```

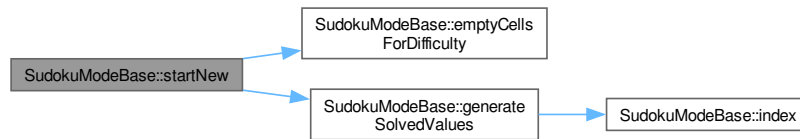
Rozpoczyna nową rozgrywkę.

Parametry

<i>difficulty</i>	Wybrany poziom trudności.
-------------------	---------------------------

Implementuje [GameMode](#).

Oto graf wywołań dla tej funkcji:



6.12.4 Dokumentacja atrybutów składowych

6.12.4.1 boxSize

```
int SudokuModeBase::boxSize [protected]
```

6.12.4.2 difficulty

```
Difficulty SudokuModeBase::difficulty [protected]
```

6.12.4.3 fixedFields

```
std::vector<int> SudokuModeBase::fixedFields [protected]
```

6.12.4.4 size

```
int SudokuModeBase::size [protected]
```

6.12.4.5 values

```
std::vector<int> SudokuModeBase::values [protected]
```

Dokumentacja dla tej klasy została wygenerowana z plików:

- [core/SudokuModeBase.h](#)
- [core/SudokuModeBase.cpp](#)

6.13 Dokumentacja klasy SudokuRules

Odpowiada za sprawdzanie zasad gry Sudoku.

```
#include <Sudokurules.h>
```

Statyczne metody publiczne

- static bool `isValueInRow` (const `Board` &board, int row, int value)
Sprawdza, czy wartość znajduje się w podanym wierszu.
- static bool `isValueInColumn` (const `Board` &board, int col, int value)
Sprawdza, czy wartość znajduje się w podanej kolumnie.
- static bool `isValueInBox` (const `Board` &board, int row, int col, int value)
Sprawdza, czy wartość znajduje się w bloku 3x3.
- static bool `isMoveValid` (const `Board` &board, int row, int col, int value)
Sprawdza, czy ruch jest zgodny z zasadami Sudoku.
- static bool `isBoardValid` (const `Board` &board)
Sprawdza, czy cała plansza jest poprawnie wypełniona.

6.13.1 Opis szczegółowy

Odpowiada za sprawdzanie zasad gry Sudoku.

Klasa udostępnia metody pomocnicze do sprawdzania, czy wartości na planszy spełniają reguły Sudoku.

6.13.2 Dokumentacja funkcji składowych

6.13.2.1 `isBoardValid()`

```
bool SudokuRules::isBoardValid (
    const Board & board) [static]
```

Sprawdza, czy cała plansza jest poprawnie wypełniona.

Metoda weryfikuje, czy plansza nie zawiera pustych pól oraz czy żadna wartość nie powtarza się w tym samym wierszu, kolumnie ani bloku 3x3.

Parametry

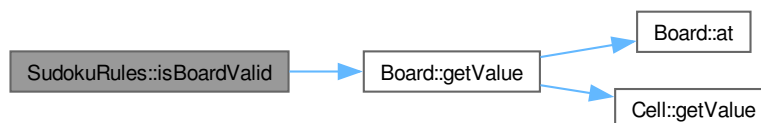
<code>board</code>	Plansza Sudoku do sprawdzenia.
--------------------	--------------------------------

Zwraca

true jeśli plansza jest pełna i zgodna z zasadami Sudoku.

false jeśli plansza zawiera puste pola lub łamie zasady Sudoku.

Oto graf wywołań dla tej funkcji:



Oto graf wywołań tej funkcji:



6.13.2.2 isMoveValid()

```

bool SudokuRules::isMoveValid (
    const Board & board,
    int row,
    int col,
    int value) [static]
  
```

Sprawdza, czy ruch jest zgodny z zasadami Sudoku.

Parametry

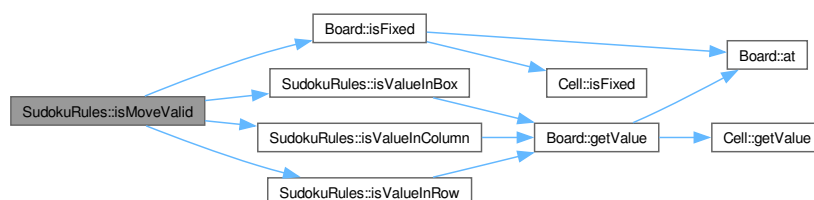
<i>board</i>	Plansza Sudoku.
<i>row</i>	Numer wiersza.
<i>col</i>	Numer kolumny.
<i>value</i>	Wartość do wpisania.

Zwraca

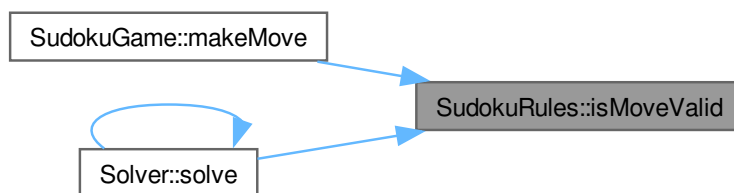
true jeśli ruch jest poprawny.

false jeśli ruch łamie zasady Sudoku.

Oto graf wywołań dla tej funkcji:



Oto graf wywoływań tej funkcji:



6.13.2.3 isValueInBox()

```

bool SudokuRules::isValueInBox (
    const Board & board,
    int row,
    int col,
    int value) [static]
  
```

Sprawdza, czy wartość znajduje się w bloku 3x3.

Parametry

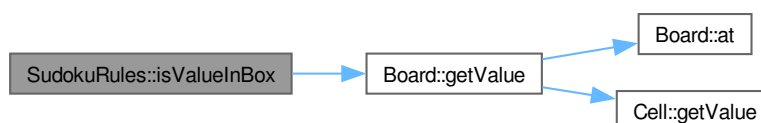
<i>board</i>	Plansza Sudoku.
<i>row</i>	Numer wiersza pola należącego do bloku.
<i>col</i>	Numer kolumny pola należącego do bloku.
<i>value</i>	Sprawdzana wartość.

Zwraca

true jeśli wartość występuje w bloku.

false jeśli wartość nie występuje w bloku.

Oto graf wywołań dla tej funkcji:



Oto graf wywołań tej funkcji:



6.13.2.4 isValueInColumn()

```

bool SudokuRules::isValueInColumn (
    const Board & board,
    int col,
    int value) [static]
  
```

Sprawdza, czy wartość znajduje się w podanej kolumnie.

Parametry

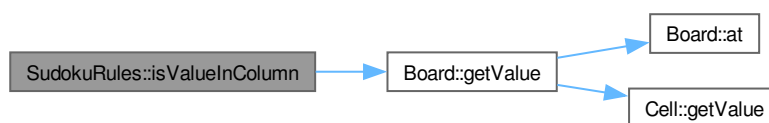
<i>board</i>	Plansza Sudoku.
<i>col</i>	Numer kolumny.
<i>value</i>	Sprawdzana wartość.

Zwraca

true jeśli wartość występuje w kolumnie.

false jeśli wartość nie występuje w kolumnie.

Oto graf wywołań dla tej funkcji:



Oto graf wywołań tej funkcji:



6.13.2.5 isValueInRow()

```
bool SudokuRules::isValueInRow (
    const Board & board,
    int row,
    int value) [static]
```

Sprawdza, czy wartość znajduje się w podanym wierszu.

Parametry

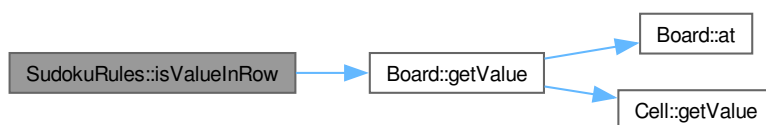
<i>board</i>	Plansza Sudoku.
<i>row</i>	Numer wiersza.
<i>value</i>	Sprawdzana wartość.

Zwraca

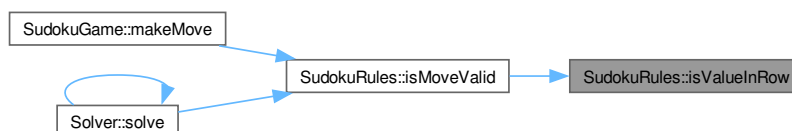
true jeśli wartość występuje w wierszu.

false jeśli wartość nie występuje w wierszu.

Oto graf wywołań dla tej funkcji:



Oto graf wywoływań tej funkcji:



Dokumentacja dla tej klasy została wygenerowana z plików:

- [core/Sudokurules.h](#)
- [core/Sudokurules.cpp](#)

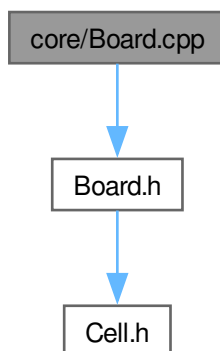
Rozdział 7

Dokumentacja plików

7.1 Dokumentacja pliku core/Board.cpp

```
#include "Board.h"
```

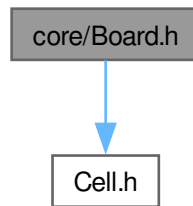
Wykres zależności załączania dla Board.cpp:



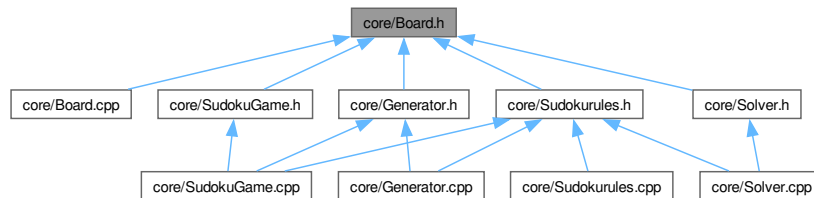
7.2 Dokumentacja pliku core/Board.h

```
#include "Cell.h"
```

Wykres zależności załączania dla Board.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class `Board`

Reprezentuje planszę Sudoku 9x9.

7.3 Board.h

[Idź do dokumentacji tego pliku.](#)

```

00001 #ifndef BOARD_H
00002 #define BOARD_H
00003
00004 #include "Cell.h"
00005
00013 class Board
00014 {
00015 private:
00016     Cell cells[9][9];
00020
00021 public:
00022
00023     static const int SIZE = 9;
00027
00028     static const int BOX_SIZE = 3;
00032
00033     Board();
00037
00038     Cell& at(int row, int col);
00046
00047
  
```

```

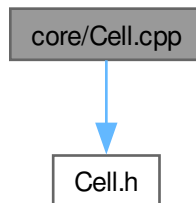
00055     const Cell& at(int row, int col) const;
00056
00064     Cell& operator()(int row, int col);
00065
00073     const Cell& operator()(int row, int col) const;
00074
00082     int getValue(int row, int col) const;
00083
00091     void setValue(int row, int col, int value);
00092
00101     bool isEmpty(int row, int col) const;
00102
00111     bool isFixed(int row, int col) const;
00112
00119     bool isFull() const;
00120
00126     void clear();
00127 };
00128
00129 #endif // BOARD_H

```

7.4 Dokumentacja pliku core/Cell.cpp

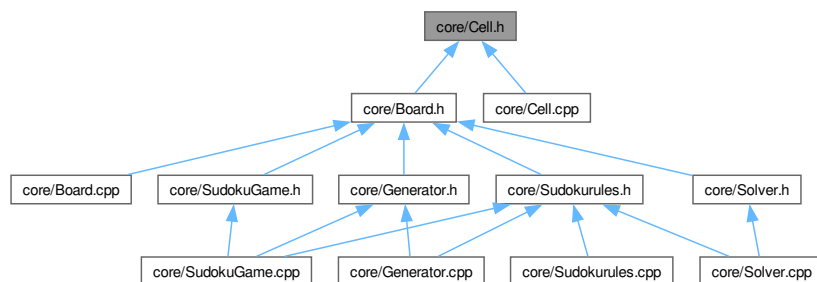
```
#include "Cell.h"
```

Wykres zależności załączania dla Cell.cpp:



7.5 Dokumentacja pliku core/Cell.h

Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class `Cell`

Reprezentuje pojedyncze pole planszy Sudoku.

7.6 Cell.h

[Idź do dokumentacji tego pliku.](#)

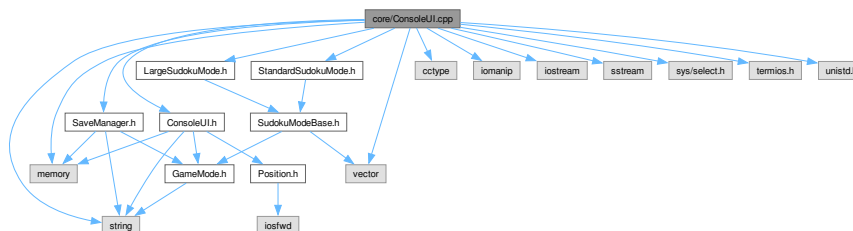
```
00001 #ifndef CELL_H
00002 #define CELL_H
00003
00012 class Cell
00013 {
00014 private:
00015
00022     int value;
00023
00030     bool fixed;
00031
00032 public:
00033
00037     Cell();
00038
00045     Cell(int value, bool fixed);
00046
00052     int getValue() const;
00053
00059     void setValue(int value);
00060
00067     bool isFixed() const;
00068
00074     void setFixed(bool fixed);
00075
00084     bool isEmpty() const;
00085
00093     void clear();
00094 };
00095
00096 #endif // CELL_H
```

7.7 Dokumentacja pliku core/ConsoleUI.cpp

```
#include "ConsoleUI.h"
#include "LargeSudokuMode.h"
#include "SaveManager.h"
#include "StandardSudokuMode.h"
#include <cctype>
#include <iomanip>
#include <iostream>
#include <memory>
#include <sstream>
#include <string>
#include <sys/select.h>
#include <termios.h>
#include <unistd.h>
```

```
#include <vector>
```

Wykres zależności załączania dla ConsoleUI.cpp:



7.8 Dokumentacja pliku core/ConsoleUI.h

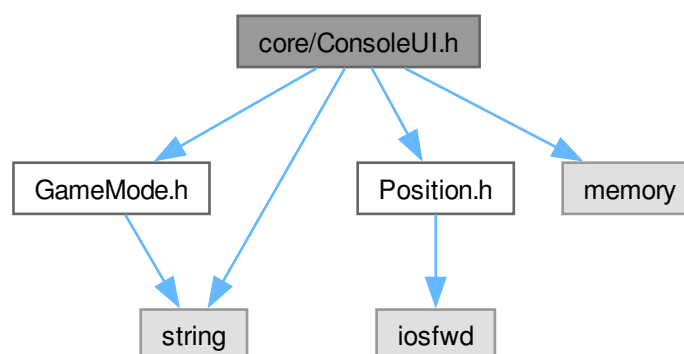
```
#include "GameMode.h"
```

```
#include "Position.h"
```

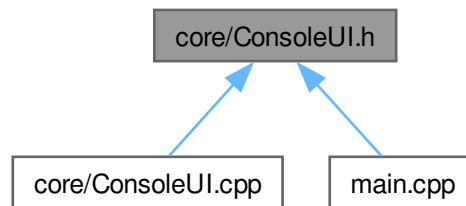
```
#include <memory>
```

```
#include <string>
```

Wykres zależności załączania dla ConsoleUI.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class `ConsoleUI`

Obsługuje menu i interaktywny terminalowy interfejs gry Sudoku.

7.9 ConsoleUI.h

[Idź do dokumentacji tego pliku.](#)

```
00001 #ifndef CONSOLEUI_H
00002 #define CONSOLEUI_H
00003
00004 #include "GameMode.h"
00005 #include "Position.h"
00006
00007 #include <memory>
00008 #include <string>
00009
00018 class ConsoleUI
00019 {
00020 private:
00021
00025     Position selectedPosition;
00026
00030     std::string statusMessage;
00031
00035     Difficulty selectedDifficulty;
00036
00040     int selectedBoxSize;
00041
00045     std::unique_ptr<GameMode> game;
00046
00050     int pendingLeadingDigit;
00051
00057     std::unique_ptr<GameMode> createSelectedMode() const;
00058
00064     void renderMenu(int selectedOption) const;
00065
00069     void runMenu();
00070
00074     void renderGame() const;
00075
00079     void runGame();
00080
00087     void moveSelection(int rowDelta, int colDelta);
00088
00096     void handleDigit(int value);
00097
00103     void enterValue(int value);
00104
00108     void clearSelectedCell();
00109
00113     void saveCurrentGame();
```



```

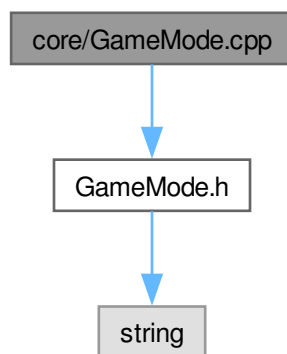
00114
00118     void startNewGame();
00119
00125     bool loadSavedGame();
00126
00127 public:
00128
00132     ConsoleUI();
00133
00137     void run();
00138 };
00139
00140 #endif // CONSOLEUI_H

```

7.10 Dokumentacja pliku core/GameMode.cpp

```
#include "GameMode.h"
```

Wykres zależności załączania dla GameMode.cpp:



Funkcje

- `std::string difficultyToString (Difficulty difficulty)`
Zamienia poziom trudności na tekst.
- `Difficulty difficultyFromString (const std::string &text)`
Zamienia tekst z pliku zapisu na poziom trudności.

7.10.1 Dokumentacja funkcji

7.10.1.1 difficultyFromString()

```

Difficulty difficultyFromString (
    const std::string & text)

```

Zamienia tekst z pliku zapisu na poziom trudności.

Parametry

<i>text</i>	Tekstowa nazwa poziomu.
-------------	-------------------------

Zwraca

Poziom trudności.

Oto graf wywoływań tej funkcji:

**7.10.1.2 difficultyToString()**

```
std::string difficultyToString (  
    Difficulty difficulty)
```

Zamienia poziom trudności na tekst.

Parametry

<i>difficulty</i>	Poziom trudności.
-------------------	-------------------

Zwraca

Nazwa poziomu trudności.

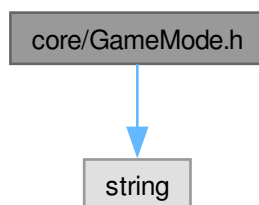
Oto graf wywoływań tej funkcji:



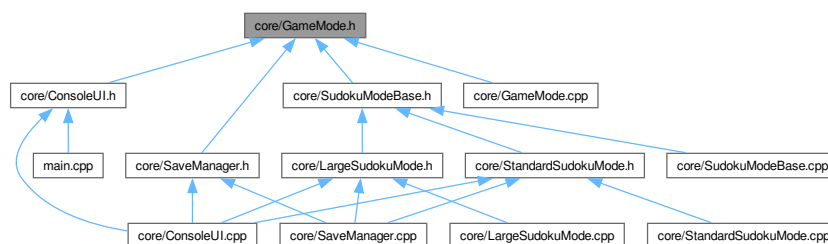
7.11 Dokumentacja pliku core/GameMode.h

```
#include <string>
```

Wykres zależności załączania dla GameMode.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class [GameMode](#)
Abstrakcyjna baza dla trybów gry.

Wyliczenia

- enum class [Difficulty](#) { [Easy](#) , [Medium](#) , [Hard](#) }
Określa poziom trudności rozgrywki.

Funkcje

- std::string [difficultyToString](#) ([Difficulty](#) difficulty)
Zamienia poziom trudności na tekst.
- [Difficulty](#) [difficultyFromString](#) (const std::string &text)
Zamienia tekst z pliku zapisu na poziom trudności.

7.11.1 Dokumentacja typów wyliczanych

7.11.1.1 Difficulty

```
enum class Difficulty [strong]
```

Określa poziom trudności rozgrywki.

Wartości wyliczeń

Easy	
Medium	
Hard	

7.11.2 Dokumentacja funkcji

7.11.2.1 difficultyFromString()

```
Difficulty difficultyFromString (  
    const std::string & text)
```

Zamienia tekst z pliku zapisu na poziom trudności.

Parametry

<i>text</i>	Tekstowa nazwa poziomu.
-------------	-------------------------

Zwraca

Poziom trudności.

Oto graf wywoływań tej funkcji:



7.11.2.2 difficultyToString()

```
std::string difficultyToString (
    Difficulty difficulty)
```

Zamienia poziom trudności na tekst.

Parametry

<i>difficulty</i>	Poziom trudności.
-------------------	-------------------

Zwraca

Nazwa poziomu trudności.

Oto graf wywoływań tej funkcji:



7.12 GameMode.h

[Idź do dokumentacji tego pliku.](#)

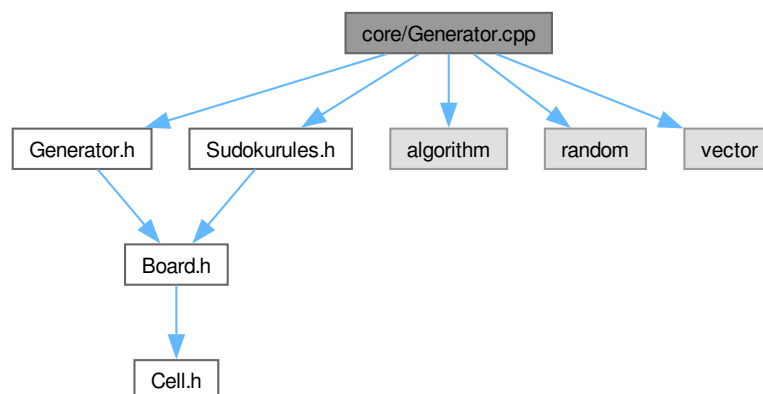
```
00001 #ifndef GAMEMODE_H
00002 #define GAMEMODE_H
00003
00004 #include <string>
00005
00010 enum class Difficulty
00011 {
00012     Easy,
00013     Medium,
00014     Hard
00015 };
00016
00023 std::string difficultyToString(Difficulty difficulty);
00024
00031 Difficulty difficultyFromString(const std::string& text);
00032
00040 class GameMode
00041 {
00042 public:
00046     virtual ~GameMode() = default;
00047
00053     virtual void startNew(Difficulty difficulty) = 0;
00054
00060     virtual std::string getName() const = 0;
00061
00067     virtual Difficulty getDifficulty() const = 0;
00068
00074     virtual int getSize() const = 0;
00075
00081     virtual int getBoxSize() const = 0;
00082
00088     virtual int getMaxValue() const = 0;
00089
00097     virtual int getValue(int row, int col) const = 0;
```

```
00098
00106     virtual bool isFixed(int row, int col) const = 0;
00107
00116     virtual bool makeMove(int row, int col, int value) = 0;
00117
00125     virtual bool clearCell(int row, int col) = 0;
00126
00132     virtual bool isCompleted() const = 0;
00133
00142     virtual bool loadState(const int* values, const int* fixed, Difficulty difficulty) = 0;
00143 };
00144
00145 #endif // GAMEMODE_H
```

7.13 Dokumentacja pliku core/Generator.cpp

```
#include "Generator.h"
#include "SudokuRules.h"
#include <algorithm>
#include <random>
#include <vector>
```

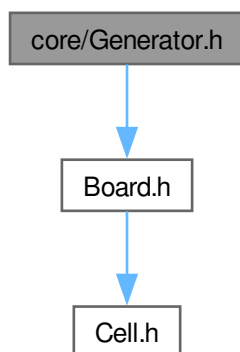
Wykres zależności załączania dla Generator.cpp:



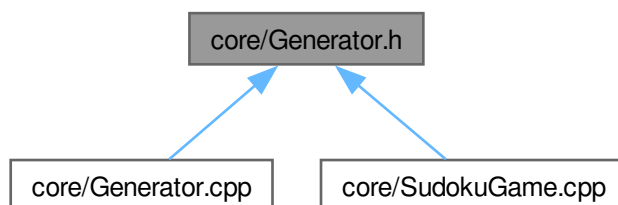
7.14 Dokumentacja pliku core/Generator.h

```
#include "Board.h"
```

Wykres zależności załączania dla Generator.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class `Generator`

Odpowiada za generowanie plansz Sudoku.

7.15 Generator.h

[Idź do dokumentacji tego pliku.](#)

```
00001 #ifndef GENERATOR_H
00002 #define GENERATOR_H
00003
00004 #include "Board.h"
```

```

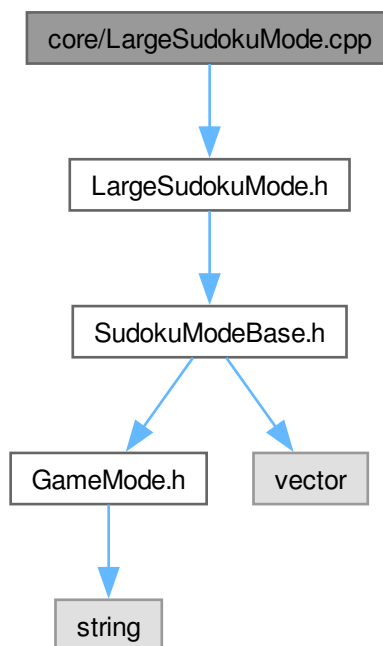
00005
00013 class Generator
00014 {
00015 public:
00016
00023     Board generatePuzzle(int emptyCells);
00024
00025 private:
00026
00032     Board generateSolvedBoard();
00033
00045     bool fillBoard(Board& board);
00046
00053     void removeCells(Board& board, int emptyCells);
00054
00060     void markFixedCells(Board& board);
00061 };
00062
00063 #endif // GENERATOR_H

```

7.16 Dokumentacja pliku core/LargeSudokuMode.cpp

```
#include "LargeSudokuMode.h"
```

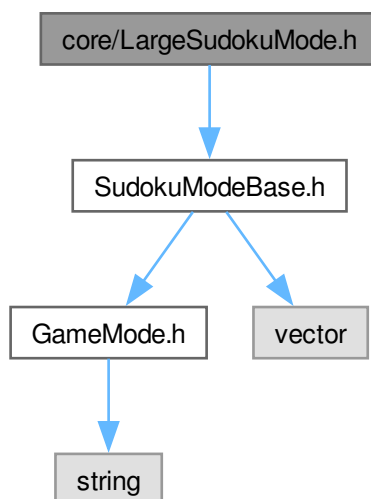
Wykres zależności załączania dla LargeSudokuMode.cpp:



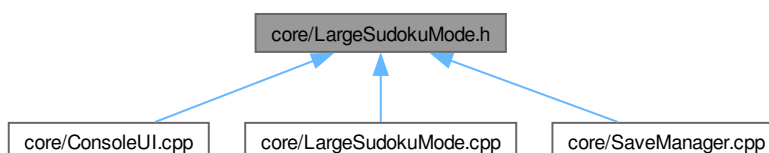
7.17 Dokumentacja pliku core/LargeSudokuMode.h

```
#include "SudokuModeBase.h"
```

Wykres zależności załączania dla LargeSudokuMode.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class [LargeSudokuMode](#)

Rozszerzone Sudoku z blokami 4x4 i planszą 16x16.

7.18 LargeSudokuMode.h

[Idź do dokumentacji tego pliku.](#)

```
00001 #ifndef LARGESUDOKUMODE_H
00002 #define LARGESUDOKUMODE_H
```

```

00003
00004 #include "SudokuModeBase.h"
00005
00010 class LargeSudokuMode : public SudokuModeBase
00011 {
00012 public:
00016     LargeSudokuMode();
00017
00018     std::string getName() const override;
00019 };
00020
00021 #endif // LARGESUDOKUMODE_H

```

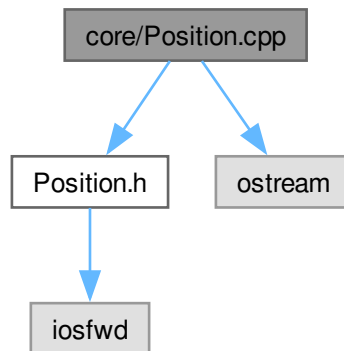
7.19 Dokumentacja pliku core/Position.cpp

```

#include "Position.h"
#include <ostream>

```

Wykres zależności załączania dla Position.cpp:



Funkcje

- `std::ostream & operator<< (std::ostream &stream, const Position &position)`

7.19.1 Dokumentacja funkcji

7.19.1.1 `operator<<()`

```

std::ostream & operator<< (
    std::ostream & stream,
    const Position & position)

```

Funkcja jest zaprzyjaźniona z klasą [Position](#), dlatego może odczytać prywatne pola `row` i `col` bez używania getterów.

Parametry

<i>stream</i>	Strumień wyjściowy.
<i>position</i>	Pozycja do wypisania.

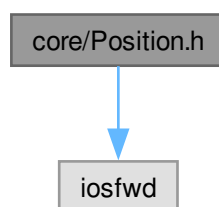
Zwraca

Referencja do strumienia wyjściowego.

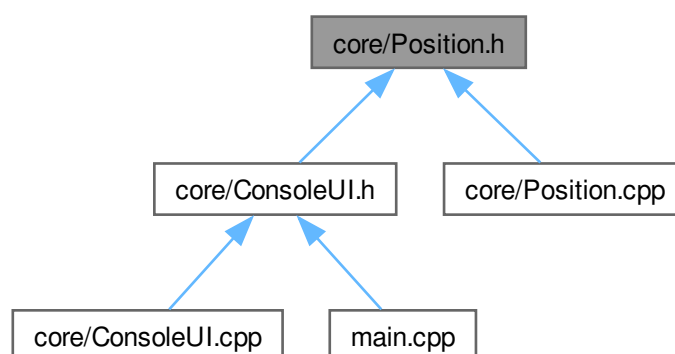
7.20 Dokumentacja pliku core/Position.h

```
#include <iosfwd>
```

Wykres zależności załączania dla Position.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:

**Komponenty**

- class [Position](#)

Reprezentuje położenie pola na planszy.

7.21 Position.h

[Idź do dokumentacji tego pliku.](#)

```

00001 #ifndef POSITION_H
00002 #define POSITION_H
00003
00004 #include <iosfwd>
00005
00014 class Position
00015 {
00016 private:
00017     int row;
00018     int col;
00019
00020 public:
00027     Position(int row = 0, int col = 0);
00028
00034     int getRow() const;
00035
00041     int getCol() const;
00042
00049     void set(int newRow, int newCol);
00050
00057     bool operator==(const Position& other) const;
00058
00065     bool operator!=(const Position& other) const;
00066
00077     friend std::ostream& operator<<(std::ostream& stream, const Position& position);
00078 };
00079
00080 #endif // POSITION_H

```

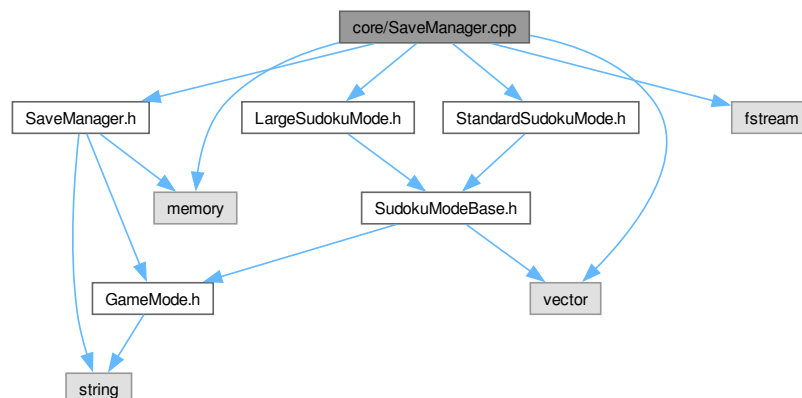
7.22 Dokumentacja pliku core/SaveManager.cpp

```

#include "SaveManager.h"
#include "LargeSudokuMode.h"
#include "StandardSudokuMode.h"
#include <fstream>
#include <memory>
#include <vector>

```

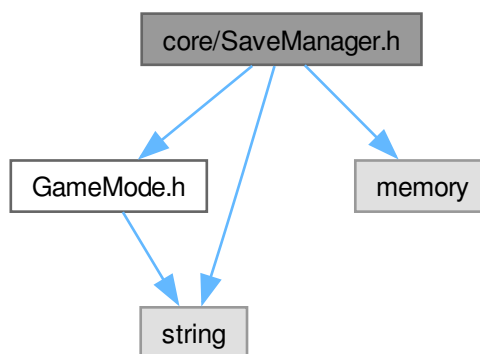
Wykres zależności załączania dla SaveManager.cpp:



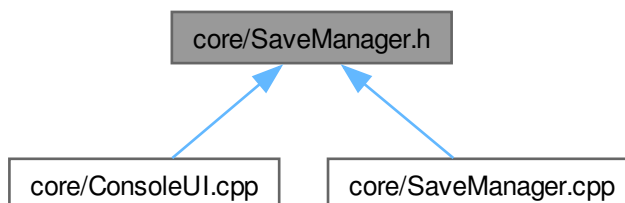
7.23 Dokumentacja pliku core/SaveManager.h

```
#include "GameMode.h"  
#include <memory>  
#include <string>
```

Wykres zależności załączania dla SaveManager.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class [SaveManager](#)

Zapisuje i odczytuje stan gry z pliku tekstowego.

7.24 SaveManager.h

[Idź do dokumentacji tego pliku.](#)

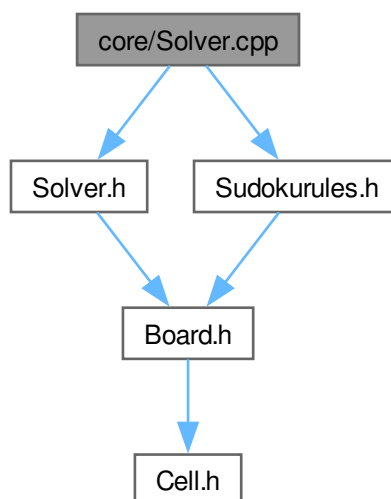
```
00001 #ifndef SAVEMANAGER_H
```

```
00002 #define SAVEMANAGER_H
00003
00004 #include "GameMode.h"
00005
00006 #include <memory>
00007 #include <string>
00008
00013 class SaveManager
00014 {
00015 public:
00019     static const char* DEFAULT_SAVE_FILE;
00020
00028     static bool save(const GameMode& game, const std::string& fileName = DEFAULT_SAVE_FILE);
00029
00036     static std::unique_ptr<GameMode> load(const std::string& fileName = DEFAULT_SAVE_FILE);
00037 };
00038
00039 #endif // SAVEMANAGER_H
```

7.25 Dokumentacja pliku core/Solver.cpp

```
#include "Solver.h"
#include "Sudokurules.h"
```

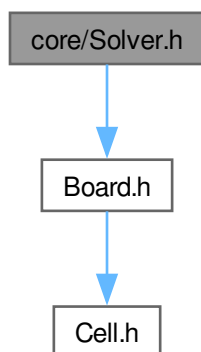
Wykres zależności załączania dla Solver.cpp:



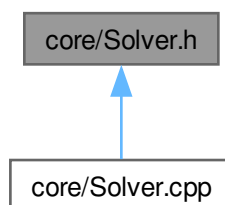
7.26 Dokumentacja pliku core/Solver.h

```
#include "Board.h"
```

Wykres zależności załączania dla Solver.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class [Solver](#)

Odpowiada za rozwiązywanie planszy Sudoku.

7.27 Solver.h

[Idź do dokumentacji tego pliku.](#)

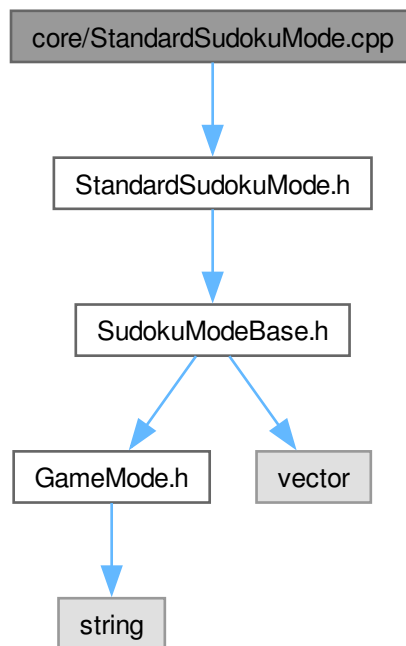
```
00001 #ifndef SOLVER_H
00002 #define SOLVER_H
00003
00004 #include "Board.h"
```

```
00005
00013 class Solver
00014 {
00015 public:
00016
00024     bool solve(Board& board);
00025
00026 private:
00027
00037     bool findEmptyCell(const Board& board, int& row, int& col) const;
00038 };
00039
00040 #endif // SOLVER_H
```

7.28 Dokumentacja pliku core/StandardSudokuMode.cpp

```
#include "StandardSudokuMode.h"
```

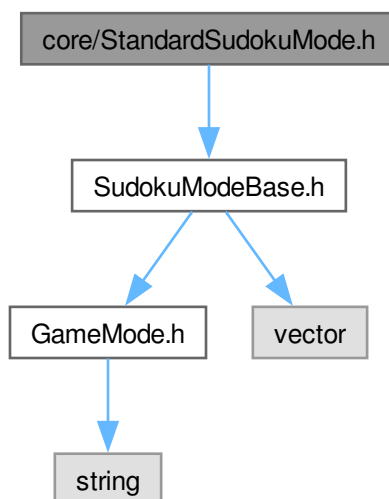
Wykres zależności załączania dla StandardSudokuMode.cpp:



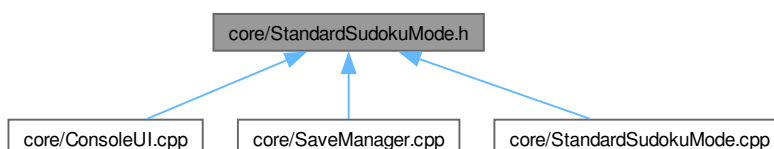
7.29 Dokumentacja pliku core/StandardSudokuMode.h

```
#include "SudokuModeBase.h"
```

Wykres zależności załączania dla StandardSudokuMode.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class [StandardSudokuMode](#)

Klasyczne Sudoku z blokami 3x3 i planszą 9x9.

7.30 StandardSudokuMode.h

[Idź do dokumentacji tego pliku.](#)

```

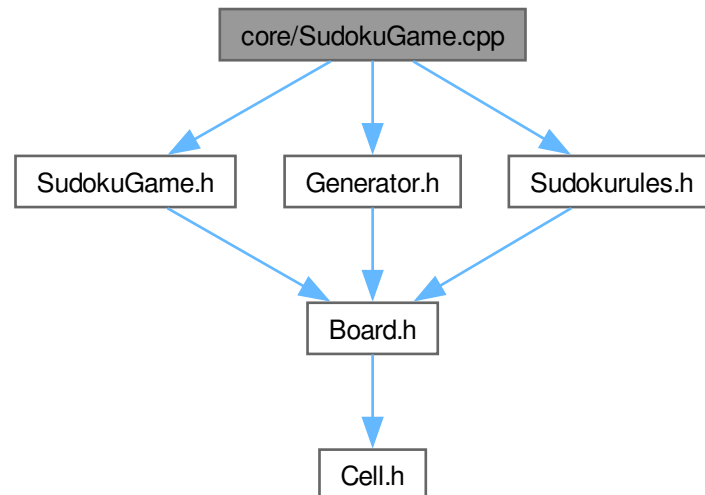
00001 #ifndef STANDARDSUDOKUMODE_H
00002 #define STANDARDSUDOKUMODE_H
00003
  
```

```
00004 #include "SudokuModeBase.h"
00005
00010 class StandardSudokuMode : public SudokuModeBase
00011 {
00012 public:
00016     StandardSudokuMode();
00017
00018     std::string getName() const override;
00019 };
00020
00021 #endif // STANDARDSUDOKUMODE_H
```

7.31 Dokumentacja pliku core/SudokuGame.cpp

```
#include "SudokuGame.h"
#include "Generator.h"
#include "Sudokurules.h"
```

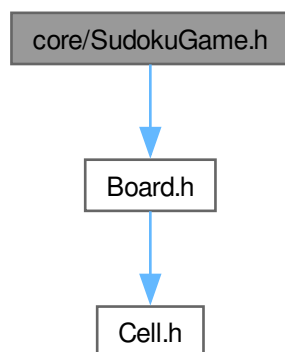
Wykres zależności załączania dla SudokuGame.cpp:



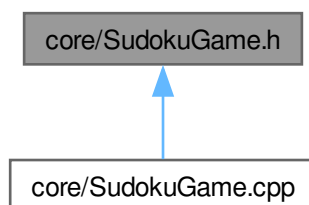
7.32 Dokumentacja pliku core/SudokuGame.h

```
#include "Board.h"
```

Wykres zależności załączania dla SudokuGame.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class [SudokuGame](#)

Zarządza aktualną rozgrywką Sudoku.

7.33 SudokuGame.h

[Idź do dokumentacji tego pliku.](#)

```
00001 #ifndef SUDOKUGAME_H
00002 #define SUDOKUGAME_H
00003
00004 #include "Board.h"
```

```

00005
00014 class SudokuGame
00015 {
00016 private:
00017
00021     Board board;
00022
00023 public:
00024
00028     SudokuGame();
00029
00035     void newGame(int emptyCells);
00036
00051     bool makeMove(int row, int col, int value);
00052
00061     bool clearCell(int row, int col);
00062
00072     bool isCompleted() const;
00073
00079     const Board& getBoard() const;
00080 };
00081
00082 #endif // SUDOKUGAME_H

```

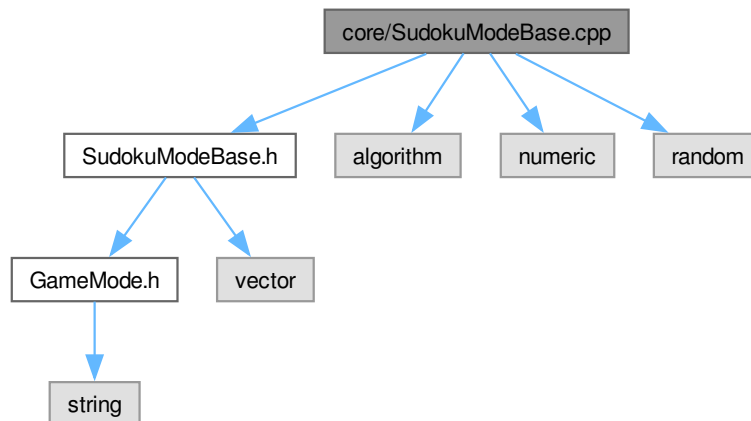
7.34 Dokumentacja pliku core/SudokuModeBase.cpp

```

#include "SudokuModeBase.h"
#include <algorithm>
#include <numeric>
#include <random>

```

Wykres zależności załączania dla SudokuModeBase.cpp:



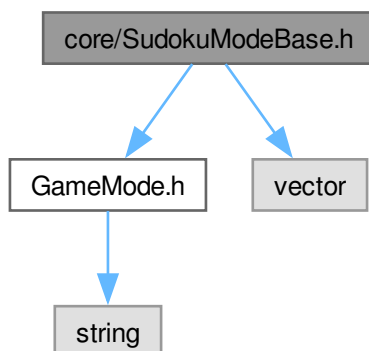
7.35 Dokumentacja pliku core/SudokuModeBase.h

```

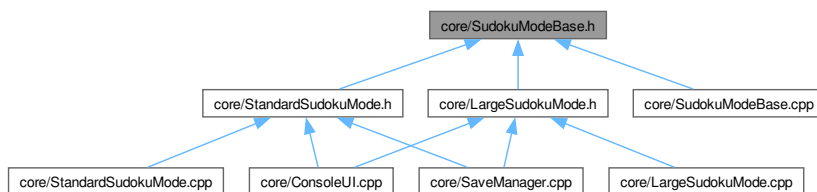
#include "GameMode.h"
#include <vector>

```

Wykres zależności załączania dla SudokuModeBase.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class [SudokuModeBase](#)

Wspólna baza dla konkretnych wariantów Sudoku.

7.36 SudokuModeBase.h

[Idź do dokumentacji tego pliku.](#)

```

00001 #ifndef SUDOKUMODEBASE_H
00002 #define SUDOKUMODEBASE_H
00003
00004 #include "GameMode.h"
00005
00006 #include <vector>
00007
00016 class SudokuModeBase : public GameMode
00017 {
00018 protected:
00019     int boxSize;
00020     int size;
00021     Difficulty difficulty;
00022     std::vector<int> values;
00023     std::vector<int> fixedFields;
  
```

```

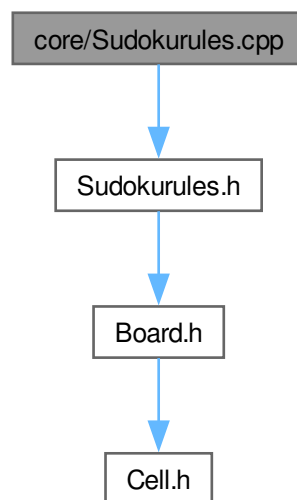
00024
00032     int index(int row, int col) const;
00033
00041     bool isInside(int row, int col) const;
00042
00049     int emptyCellsForDifficulty(Difficulty difficulty) const;
00050
00056     std::vector<int> generateSolvedValues() const;
00057
00066     bool canPlaceValue(int row, int col, int value) const;
00067
00068 public:
00074     explicit SudokuModeBase(int boxSize);
00075
00076     void startNew(Difficulty difficulty) override;
00077     Difficulty getDifficulty() const override;
00078     int getSize() const override;
00079     int getBoxSize() const override;
00080     int getMaxValue() const override;
00081     int getValue(int row, int col) const override;
00082     bool isFixed(int row, int col) const override;
00083     bool makeMove(int row, int col, int value) override;
00084     bool clearCell(int row, int col) override;
00085     bool isCompleted() const override;
00086     bool loadState(const int* values, const int* fixed, Difficulty difficulty) override;
00087 };
00088
00089 #endif // SUDOKUMODEBASE_H

```

7.37 Dokumentacja pliku core/Sudokurules.cpp

#include "Sudokurules.h"

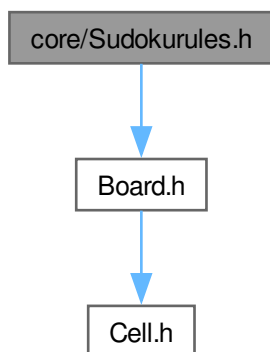
Wykres zależności załączania dla Sudokurules.cpp:



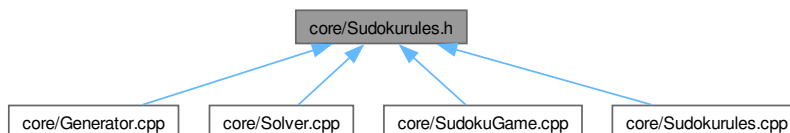
7.38 Dokumentacja pliku core/Sudokurules.h

```
#include "Board.h"
```

Wykres zależności załączania dla Sudokurules.h:



Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- class `SudokuRules`

Odpowiada za sprawdzanie zasad gry Sudoku.

7.39 Sudokurules.h

[Idź do dokumentacji tego pliku.](#)

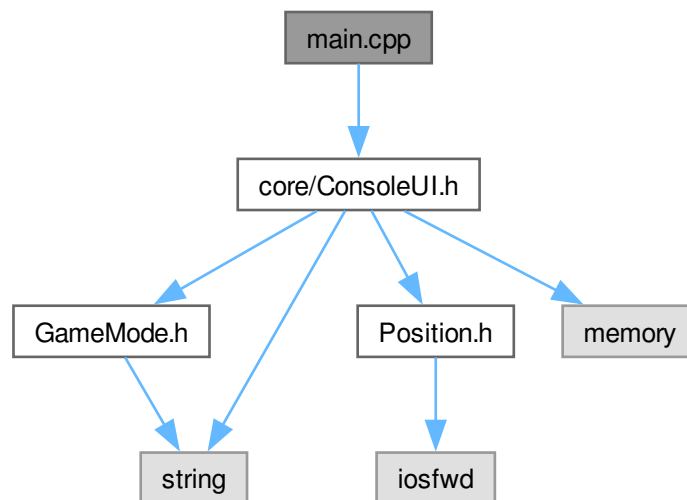
```
00001 #ifndef SUDOKURULES_H
00002 #define SUDOKURULES_H
00003
00004 #include "Board.h"
00005
00013 class SudokuRules
00014 {
00015 public:
00016
00026     static bool isValueInRow(const Board& board, int row, int value);
00027
```

```
00037     static bool isValueInColumn(const Board& board, int col, int value);
00038
00049     static bool isValueInBox(const Board& board, int row, int col, int value);
00050
00061     static bool isMoveValid(const Board& board, int row, int col, int value);
00062
00074     static bool isBoardValid(const Board& board);
00075 };
00076
00077 #endif // SUDOKURULES_H
```

7.40 Dokumentacja pliku main.cpp

```
#include "core/ConsoleUI.h"
```

Wykres zależności załączania dla main.cpp:



Funkcje

- int `main` ()

7.40.1 Dokumentacja funkcji

7.40.1.1 `main()`

```
int main ()
```


Oto graf wywołań dla tej funkcji:



Indeks

- ~GameMode
 - GameMode, [25](#)
- at
 - Board, [12](#)
- Board, [11](#)
 - at, [12](#)
 - Board, [12](#)
 - BOX_SIZE, [19](#)
 - clear, [13](#)
 - getValue, [13](#)
 - isEmpty, [14](#)
 - isFixed, [15](#)
 - isFull, [16](#)
 - operator(), [16](#), [17](#)
 - setValue, [18](#)
 - SIZE, [19](#)
- BOX_SIZE
 - Board, [19](#)
- boxSize
 - SudokuModeBase, [60](#)
- canPlaceValue
 - SudokuModeBase, [51](#)
- Cell, [19](#)
 - Cell, [20](#)
 - clear, [20](#)
 - getValue, [20](#)
 - isEmpty, [21](#)
 - isFixed, [21](#)
 - setFixed, [22](#)
 - setValue, [22](#)
- clear
 - Board, [13](#)
 - Cell, [20](#)
- clearCell
 - GameMode, [26](#)
 - SudokuGame, [46](#)
 - SudokuModeBase, [52](#)
- ConsoleUI, [23](#)
 - ConsoleUI, [23](#)
 - run, [24](#)
- core/Board.cpp, [67](#)
- core/Board.h, [67](#), [68](#)
- core/Cell.cpp, [69](#)
- core/Cell.h, [69](#), [70](#)
- core/ConsoleUI.cpp, [70](#)
- core/ConsoleUI.h, [71](#), [72](#)
- core/GameMode.cpp, [73](#)
- core/GameMode.h, [75](#), [77](#)
- core/Generator.cpp, [78](#)
- core/Generator.h, [79](#)
- core/LargeSudokuMode.cpp, [80](#)
- core/LargeSudokuMode.h, [81](#)
- core/Position.cpp, [82](#)
- core/Position.h, [83](#), [84](#)
- core/SaveManager.cpp, [84](#)
- core/SaveManager.h, [85](#)
- core/Solver.cpp, [86](#)
- core/Solver.h, [87](#)
- core/StandardSudokuMode.cpp, [88](#)
- core/StandardSudokuMode.h, [89](#)
- core/SudokuGame.cpp, [90](#)
- core/SudokuGame.h, [91](#)
- core/SudokuModeBase.cpp, [92](#)
- core/SudokuModeBase.h, [92](#), [93](#)
- core/Sudokurules.cpp, [94](#)
- core/Sudokurules.h, [95](#)
- DEFAULT_SAVE_FILE
 - SaveManager, [40](#)
- Difficulty
 - GameMode.h, [76](#)
- difficulty
 - SudokuModeBase, [60](#)
- difficultyFromString
 - GameMode.cpp, [73](#)
 - GameMode.h, [76](#)
- difficultyToString
 - GameMode.cpp, [74](#)
 - GameMode.h, [76](#)
- Dokumentacja katalogu core, [9](#)
- Easy
 - GameMode.h, [76](#)
- emptyCellsForDifficulty
 - SudokuModeBase, [52](#)
- fixedFields
 - SudokuModeBase, [60](#)
- GameMode, [24](#)
 - ~GameMode, [25](#)
 - clearCell, [26](#)
 - getBoxSize, [26](#)
 - getDifficulty, [26](#)
 - getMaxValue, [27](#)
 - getName, [27](#)
 - getSize, [27](#)

- getValue, 28
 - isCompleted, 28
 - isFixed, 29
 - loadState, 29
 - makeMove, 30
 - startNew, 30
- GameMode.cpp
 - difficultyFromString, 73
 - difficultyToString, 74
- GameMode.h
 - Difficulty, 76
 - difficultyFromString, 76
 - difficultyToString, 76
 - Easy, 76
 - Hard, 76
 - Medium, 76
- generatePuzzle
 - Generator, 31
- generateSolvedValues
 - SudokuModeBase, 53
- Generator, 31
 - generatePuzzle, 31
- getBoard
 - SudokuGame, 46
- getBoxSize
 - GameMode, 26
 - SudokuModeBase, 54
- getCol
 - Position, 36
- getDifficulty
 - GameMode, 26
 - SudokuModeBase, 54
- getMaxValue
 - GameMode, 27
 - SudokuModeBase, 54
- getName
 - GameMode, 27
 - LargeSudokuMode, 35
 - StandardSudokuMode, 45
- getRow
 - Position, 36
- getSize
 - GameMode, 27
 - SudokuModeBase, 54
- getValue
 - Board, 13
 - Cell, 20
 - GameMode, 28
 - SudokuModeBase, 55
- Hard
 - GameMode.h, 76
- index
 - SudokuModeBase, 55
- isBoardValid
 - SudokuRules, 61
- isCompleted
 - GameMode, 28
 - SudokuGame, 46
 - SudokuModeBase, 56
- isEmpty
 - Board, 14
 - Cell, 21
- isFixed
 - Board, 15
 - Cell, 21
 - GameMode, 29
 - SudokuModeBase, 56
- isFull
 - Board, 16
- isInside
 - SudokuModeBase, 57
- isMoveValid
 - SudokuRules, 62
- isValueInBox
 - SudokuRules, 63
- isValueInColumn
 - SudokuRules, 64
- isValueInRow
 - SudokuRules, 64
- LargeSudokuMode, 32
 - getName, 35
 - LargeSudokuMode, 34
- load
 - SaveManager, 39
- loadState
 - GameMode, 29
 - SudokuModeBase, 58
- main
 - main.cpp, 96
- main.cpp, 96
 - main, 96
- makeMove
 - GameMode, 30
 - SudokuGame, 47
 - SudokuModeBase, 58
- Medium
 - GameMode.h, 76
- newGame
 - SudokuGame, 48
- operator!=
 - Position, 37
- operator<<
 - Position, 38
 - Position.cpp, 82
- operator()
 - Board, 16, 17
- operator==
 - Position, 37
- Position, 35
 - getCol, 36
 - getRow, 36

- operator!=, 37
 - operator<<, 38
 - operator==, 37
 - Position, 36
 - set, 38
- Position.cpp
 - operator<<, 82
- run
 - ConsoleUI, 24
- save
 - SaveManager, 39
- SaveManager, 39
 - DEFAULT_SAVE_FILE, 40
 - load, 39
 - save, 39
- set
 - Position, 38
- setFixed
 - Cell, 22
- setValue
 - Board, 18
 - Cell, 22
- SIZE
 - Board, 19
- size
 - SudokuModeBase, 60
- solve
 - Solver, 41
- Solver, 41
 - solve, 41
- StandardSudokuMode, 42
 - getName, 45
 - StandardSudokuMode, 44
- startNew
 - GameMode, 30
 - SudokuModeBase, 59
- SudokuGame, 45
 - clearCell, 46
 - getBoard, 46
 - isCompleted, 46
 - makeMove, 47
 - newGame, 48
 - SudokuGame, 46
- SudokuModeBase, 49
 - boxSize, 60
 - canPlaceValue, 51
 - clearCell, 52
 - difficulty, 60
 - emptyCellsForDifficulty, 52
 - fixedFields, 60
 - generateSolvedValues, 53
 - getBoxSize, 54
 - getDifficulty, 54
 - getMaxValue, 54
 - getSize, 54
 - getValue, 55
 - index, 55
 - isCompleted, 56
 - isFixed, 56
 - isInside, 57
 - loadState, 58
 - makeMove, 58
 - size, 60
 - startNew, 59
 - SudokuModeBase, 51
 - values, 60
- SudokuRules, 60
 - isBoardValid, 61
 - isMoveValid, 62
 - isValueInBox, 63
 - isValueInColumn, 64
 - isValueInRow, 64
- values
 - SudokuModeBase, 60